



Building AI Applications with Semantic Kernel and C#

Getting to Know Semantic Kernel

Understanding Semantic Kernel

AI is becoming an essential part of modern applications. Integrating it into .NET applications efficiently can be challenging. That's where Semantic Kernel comes in. It provides a powerful, flexible, and extensible way to bring AI capabilities into your .NET apps. Hello, and welcome to Building AI Applications with Semantic Kernel and C#, a course entirely dedicated to learning how to effectively make use of the Semantic Kernel library from your C# and .NET apps. I'm Gill Cleeren, and I will be your guide through this course. Throughout this course, we'll learn all there is to learn about Semantic Kernel. We'll start with the basics in this module, that is understanding what this library is and what it can be used for. So we'll look first at its core functionalities, like chat and plugins and more. I'll show you the principles using simple console applications so that we can keep the focus on learning about Semantic Kernel. But once you have enough knowledge, we'll also see how we can use it in a real application, and we'll also go further and we'll explore the more advanced concepts, such as agents, memory, and logging. And once you're done with this course, you'll have a solid understanding of the Semantic Kernel library. That is a promise. And next to using Semantic Kernel, there are other ways to integrate AI into your apps, including the OpenAI SDK and Azure AI services. I have another course here on Pluralsight that explains you the other options simply titled Building AI-enabled Applications in .NET. If you're interested in the other options, make sure to check out that course, too. So let's start at the beginning. So first we need to understand what Semantic Kernel is really all about. Well, a Semantic Kernel is at its core a library, so it's an SDK created by Microsoft. It is very lightweight and also very extensible, which means that future changes to AI models can easily be integrated. It can be used from .NET and C# applications, and just like most libraries nowadays, it is also open source. The goal of Semantic Kernel is to abstract away the complexities of working with different AI models. So, in essence, Semantic Kernel acts as a middleware, providing an easy way to integrate different AI models into your applications. Without it, each AI provider has its own API with different request formats, authentication methods, and response handling. This means that if you wanted to switch AI providers, you'd have to rewrite large portions of your applications. Semantic Kernel removed that complexity by standardizing interactions across different models, allowing you to use them in a unified way. We'll see later that, for example, using OpenAI or Claude as our model requires just a tiny change to point it to a different



model. But all other interactions will stay the same. Through the library, most common tasks like text generation, chat completion, image generation, and a lot more are supported, as said, in a unified way. And finally, while the library is primarily designed for .NET, so used with C#, you can also use it in Java and Python. This makes it flexible for teams that work with multiple programming languages or want to integrate AI into existing applications built in different text stacks. You only have to learn about Semantic Kernel once. So to summarize, I see three big benefits in using Semantic Kernel to integrate AI into your .NET apps. As said, Semantic Kernel provides an abstraction layer. That's a big plus, since it takes away the need we're seeing to understand how the different AI models work. Abstraction gives us a higher-level unified way to interact with different AI models. Secondly, integration. It comes indeed with support for several core AI model features already integrated, like chat, text, and image generation. And finally, extensibility. Semantic Kernel is built to be flexible. You can add plugins and extend it with additional AI services very easily. Now, don't worry if you don't know what plugins are. You soon will.

Key Components of Semantic Kernel

Now that we know what Semantic Kernel is all about, let's start getting to know its key components. The Semantic Kernel library consists out of a number of components which we can use separately or in combination. We'll look at this in more detail, a lot more detail, in fact, throughout this course. Of course, there's the kernel. That's the one we'll start with. There are service connectors we need to look at. We'll look at these already in this module. Then there's the very important building block called functions and plugins. There's prompts and prompt templates. We can use memory, and we can use filters. I think those are different categories of components that we have and can use in Semantic Kernel. Let us dive into this in more detail. Judging from the name, the kernel in Semantic Kernel is obviously an important one. It is the central component, acting as an orchestrator between our app and the AI models. It is the kernel that handles the communication and ensures that the replication interacts with these services in a unified way. One of its key responsibilities is managing services and plugins. With services, I mean AI models and logging mechanisms. While plugins extend AI functionality by integrating additional capabilities, such as calling external APIs or retrieving data from databases, the kernel brings these elements together, ensuring they're all available when needed. The kernel also works as a dependency injection container, and by this I mean that it'll hold all AI services, plugins, and more that application requires and makes it accessible when needed. So we do an AI task like executing a prompt, the kernel will ensure that the right components are available and are used efficiently. Now getting started using the kernel through Semantic Kernel isn't very difficult. First, you'll need to install the NuGet package. Now, since Semantic Kernel is a .NET library, the easiest way to add it to your project is through NuGet, Microsoft's



Package Manager. Next, you'll need to choose an AI model. Like we saw, Semantic Kernel is designed to work with multiple AI models, including OpenAI's GPT, Azure OpenAI, and other AI services. And depending on your use case, you might use a text-based model for chat completion or an image generation model. And finally, you'll meet an API key. AI services require authentication, and an API key is what allows your application to securely communicate with them. Each provider issues an API key that you can configure in your application to make AI requests. We'll learn later how we can store these securely in our application. Once you have the kernel set up, the next step is to understand how it interacts with services and plugins. Let's first talk about services. Services include AI models, like chat completion, text generation, or image creation, but they also extend beyond AI. The kernel can manage additional services, such as logging, HTTP clients, or other utilities needed for your application. This follows the service provider pattern in .NET, allowing for dependency injection across different languages. Next, we have plugins. Plugins are additional components that help the kernel to perform specific tasks beyond just calling an AI model. For example, a plugin could fetch real-time data from an external API, retrieve database records, or trigger other application workflows. This makes the kernel highly extensible, as you can integrate AI-driven functionality with custom business logic. We'll learn a lot more about plugins in this and the coming modules. So by combining services and plugins, the kernel becomes the central hub for AI operations in your application. Let's now learn how we can create one. The snippet we see here initializes a new kernel instance using the `kernel.CreateBuilder` followed by the `.Build`, which basically finalizes the configuration. At this stage, the kernel itself is ready, but it's also empty. It doesn't yet have AI models or plugins connected. We can start with adding AI services using the code we see here on this snippet. Using the `AddOpenAIChatCompletion` registers a service, in this case, the OpenAI chat model with the kernel. I'm passing in two parameter values, namely the model ID, which can be, for example, GPT 4, and the API key, which is needed for authentication with the service. And with that configured kernel instance, we can now very easily create the smallest AI chat application. This next code snippet here sets up a basic console-based chatbot that continuously takes user input, sends it to the AI model, and prints the response. The most important line here is the highlighted one, using `kernel.InvokePromptAsync`, which sends the input of the user to the AI model using our kernel. Now the previous example shows how we can use Semantic Kernel to create chat completion, and that is definitely the most common thing we're using AI for right now, with LLMs being all hype. But we can plug in other AI services into the kernel. We're not limited to just using one in a kernel in our application. We can use a text-to-image service where we use the description to have the AI service generate an image. The opposite also exists and can also be used from Semantic Kernel, so that's image-to-text, where we give the model an image and it'll generate a description of what it sees. This is a service that can be used, for example, to detect anomalies. Text-to-audio, where we give the AI service a text and let it generate an audio file using your voice, can



also be used. And again, the opposite also exists, where we give the AI service a piece of audio and let it transcribe it.

Demo: Creating a First Application with Semantic Kernel

Now, it is time to see Semantic Kernel in action in a first demo. So let's start by setting up Semantic Kernel in a console application, and we'll begin with the most basic task that you would typically do in an AI model, and that is chat completion. I'm going to use a console application to focus purely on what is important and this Semantic Kernel. So for this course, I've created one solution, as you can see here, and in there I have several solution folders, one per module. And this is, of course, that first module. So we'll start with the demos in this folder here. I will need to take a look at the code inside the `GettingStartedWithSK` project. And before diving into the code, I've added the latest version of the Semantic Kernel NuGet packages, the current version at the time of this recording. You'll find the code for this demo in the `BasicsOfSk.cs` file. This file includes several methods, each highlighting a piece of functionality. And we'll start with this one here. So in this, the simplest prompt loop methods, we're setting up a basic chat completion application. The application uses an open API key, which I installed here in the `launchSettings.json`, and it allows Semantic Kernels through my application to communicate with the model. Since Semantic Kernel is, in fact, an abstraction layer, it is very straightforward to switch between models with minimal, if any, code changes. So we'll start here with the initialization of a kernel object, which serves as the primary interface to the OpenAI model through Semantic Kernel. The code calls the `kernel.create builder` to initiate building the Semantic Kernel instance. Then using the `AddOpenAIChatCompletion`, it configures the kernel to use OpenAI's chat completion model. This model requires the model name and the API key. This build method here finalizes the configuration, and that creates the kernel instance ready for use, basically. Think of this setup really as configuring a dependency injection container in .NET applications. The kernel acts like a DI container, where services are being registered, and in this case, that is the OpenAI chat completion service. Then with the kernel configured, I'm invoking here the `InvokePromptAsync` method, passing in the user's input text basically as the prompt. As if we've configured it with OpenAI, it will communicate with the OpenAI chat completions endpoint and hopefully return a response. And this method handles quite a bit of work behind the scenes. It submits the prompt, processes it, and also waits for the reply. Now, let's run this. To do that, I have this code here in the `Program.cs`. First, I'm creating a new configuration builder instance. This builder is the starting point for defining where our configuration data is going to come from. Using this line here, we set the base part to the application's current directory, and so this ensures that the configuration file is correctly located, no matter where the application runs. Next, I add the `appSettings.json` using this `.AddJsonFile` line here. And once the builder is



configured, we call this build method here to create the actual IConfiguration object. This object allows us to easily retrieve configuration values from the sources that we've set up. I'm going to here extract two values, modelName and imageModel. These are pulled from the appSettings.json file using simple key lookups. And this code will be the same throughout most of the demos of this course, so I won't be spending time on this again. So now time to run this. I don't expect miracles here. When we ask a question, we'll get a response. If we ask another one, we'll get another response. And since we're not maintaining any chat history here, every question must be self-contained. The model doesn't remember previous questions, so each prompt needs to include enough context for the model to generate an accurate response. Even though this setup is very simple, it already showcases the power of Semantic Kernel. The abstraction makes it very straightforward and clean to interact with the model, and that sets a solid foundation for more advanced scenarios. Now, one thing we saw was that there was a small delay between sending the question and getting the answer back. We don't see that the model is actually building up the answer. And so the answer is sent as one big block at the very end. We can create a streaming response using the code that we see here, which gives a better experience and works more like chat GPT and other AI models that allow us to interact. Now you see that most of the code is actually very similar here, except for the await foreach loop here, in which I'm using the kernel.InvokePromptStreamingAsync, passing in the input again. It is this method which enables the streaming of the response and processing and showing each chunk as it arrives. Using the await foreach, we asynchronously iterate over these chunks and write them to the console immediately with Console.Write, passing in that chunk here. This will give the effect of the AI typing the answer in real time. If I run the code now, we can indeed see that the response is streaming in and being shown almost immediately. And let's look at one more method here where I'm going to work with the settings of the model. First, let me explain what settings are very quickly. When we interact with AI models, settings, including temperature, max tokens, and others allow us to control how the model will behave. These settings influence the creativity, the length, and the style of the generated responses. For example, a low temperature makes responses more focused and deterministic, while a higher temperature encourages creativity and variation. Adjusting these settings gives us more control over how the AI will respond, depending on the scenario. I'm again setting up to use OpenAI, as you've already seen. And then I'm defining the first set of prompt execution settings using the KernelArguments object and I specify two key parameters. The MaxTokens is set to 500, ensuring the response doesn't exceed this limit and the temperature is set to 0, which makes the model more deterministic. At this setting, the AI will generate more predictable and straightforward answers. These settings are not passed into the model here. And next, I'm doing the same thing, but now I'm setting the temperature to 1, so we'll get a more creative answer. If we run this, we will indeed get two different stories. The first one is a bit more straightforward, and the second one, if you would read it through, is actually a bit



more creative. So through the abstraction at Semantic Kernels, we're able to work with these AI model settings also from our .NET code in a very easy way.

Demo: Generating an Image

And next to chat completion, we can add other services into the kernel. And let's try that now to include a service to generate an image. And let's look at the code inside this `GenerateBasicImage` method. First, I began creating the kernel instance, but this time I'm adding support for OpenAI's text-to-image service by using the `AddOpenAITextToImage` method. So the model name that is provided is `dall-e-3`, and the API key remains the same. Hence, I'm retrieving a text-to-image service using the same approach. Once I have the service, the method calls the `imageService.GenerateImageAsync` method, passing in the prompt, along with the desired dimensions, which is what you see here, to generate an image with the specific details and size. The prompt, as you can see, is very detailed. In fact, I had it generated by ChatGPT and I copied it in here. Now the asynchronous call returns a URL pointing to the generated image, and that is then printed to the console. I'm not downloading the image, I'm simply going to display the URL for the image. So let's give this a try and see if we can get an image generated. When the method runs, Semantic Kernel generates a link to the image. And if I copy that to my browser, then we see the generated image.

Understanding Plugins and Functions

We've now seen how services can be added, but I mentioned earlier that the other main strength of Semantic Kernel is working with plugins and functions. Let's look at this next. Large language models are incredibly powerful, but they have a key limitation. They can only work with the information they were trained on. This means that they can't fetch live data, such as the latest weather updates, book an appointment, or access company-specific databases in real time. Their knowledge is essentially frozen at their last training update. That's where function calling comes in. Function calling allows us to extend the capabilities of an LLM by giving it access to real-world data and external systems. So instead of relying solely on pre-trained information, the model can invoke functions that retrieve live data, perform actions, or interact with external APIs. Now with Semantic Kernel, enabling the use of function calling is very easy and is supported out of the box. Before we dive into creating plugins and functions with Semantic Kernel, let's take a moment to learn about the overall flow of how function calling works with an LLM. Understanding this process will help us clarify how the AI interacts with external tools and services. The first step in this process is serializing the functions. This means that Semantic Kernel packages have the functions, essentially the tools or APIs that the model can call in a way that the AI can understand. These functions serve as a bridge



between the AI and real-world data or actions. Once the available functions are civilized, the message is sent to the model. The AI receives the user's input and begins processing the request. At this stage, the model analyzes what the user is asking and determines if it can answer the question on its own using its trained knowledge or if it needs to invoke an external function to get the required information. If a function is needed, the model extracts the function name and the required parameters. For example, if the user asks for stock prices, the AI might determine that it needs to call a function named `getStockPrice` with a parameter specifying the stock symbol. Next, the kernel now invokes the function. At this point, the AI is basically stepping aside, and the external function, whether it is retrieving data from a database, calling a third-party API, but performing just any other action executes the request. Once the function completes the job, the result is sent back to the model. Retrieved data, such as a stock price, or weather update, or a database query result is returned so that the AI can use it to create its response. And finally, the kernel processes the response and delivers it back to the user. The AI takes the function's output, integrates it with existing knowledge, and generates a final natural language reply that fully answers the original request. In the context of Semantic Kernel, we talk about plugins and functions. Plugins are classes that will contain one or more functions that we are exposing to the LLM. Here you can see that we are creating a native plugin. So one containing C# methods, those are the functions. We have here a regular C# method to `GetCurrentDateAndTime`, and that'll run from within our app. But notice I've added the `KernelFunction` attribute onto it, which will ensure that this gets exposed to the kernel. It is vital that we pass in information about what the function can do so that the model, when it receives your input, knows it can look at the function we have defined. Semantic Kernel will do its job here, but we need to include that `Description` attribute here, too. Now just adding the attribute won't do anything just yet. We need to register our plugin with the kernel. There are several options here, but here I'm using the simplest one, the `ImportPluginFromType`, passing in the type that contains the plugin, so the functions we want to expose basically. We also need to enable that the functions can be invoked by the LLM, and that we are doing here using these settings. By setting `ToolCallBehavior` to `AutoInvokeKernelFunctions`, we ensure that whenever the AI determines a function is needed, the kernel automatically executes the appropriate plugin function.

Demo: Creating a Plugin with Functions

All right, now that we know what plugins are and have seen their basic structure, let's return to Visual Studio and create a plugin with a function and let the LLM invoke it. You can find the example in this first plugin class, and let's start with a very simple example, asking the model how many days there are left until Christmas. Now, while the model is smart enough to understand that it needs to know the current date to answer that question, it doesn't by default have



access to the current date because that changes constantly. So we need to provide a way for the model to figure that out. And to solve this, I'm going to extend the kernel's capabilities by importing a custom plugin called the `TimePlugin`. This is a custom type I've created as part of the application. So by calling `kernel.ImportPluginFromType`, passing in `TimePlugin`, I add the `TimePlugin` class as a new functionality within the kernel. Here's what the plugin class looks like. It contains, in this case, just one single method, the `GetCurrentDateAndTime`, which is decorated with the `KernelFunction` attribute. This marks it as a `KernelFunction` that Semantic Kernel can automatically recognize and call when relevant for the prompt. The first time we send a prompt to the model, Semantic Kernel will inform that this function is available for use. And if the model decides that it needs to use the current date, Semantic Kernel will handle the invocation of the method and pass the result back to the model, and that happens all automatically. We also have this `Description` attribute attached to the method. This `Description` helps the kernel understand the purpose of the function, making it easier to determine which functions are relevant for the given question. And the function itself is very straightforward. It calls the `DateTime.UtcNow` to return the current date and time. Now back in the `GetDaysUntilChristmas` method, we also define `OpenAIPromptExecution` settings, which set the `ToolCallBehavior` to `ToolCallBehavior.AutoInvokeKernelFunctions`. And this tells the kernel to automatically invoke any relevant functions, including `GetCurrentDateAndTime`, whenever they are needed to answer the prompt. And finally, I'm sending a prompt to the kernel again asking how many days there are still left until Christmas this year. And so using `kernel.InvokePromptAsync` along with the execution settings, the kernel can combine both the model's capabilities and the custom plugin to generate a custom answer. All right, let's run this. And there we go. In one go, we get the answer. But behind the scenes, Semantic Kernel was doing a lot of the work. Now I want to show you that our method was actually invoked. I'll put a breakpoint here and I'll run the application again. And indeed, we see that the method, the local method, is being invoked. So this approach is very simple and clean. Just by using a few attributes, we get powerful automated function calling and thus we can easily extend the model's capabilities.

Storing Keys in Your Application Securely

Before we move on, there's one more critical topic to cover, and that is securely storing API keys. When working with AI services, one of the most important things to consider is how to securely store your API keys. These keys serve as authentication credentials for your app, allowing it to interact with AI services. API keys are issued by the service provider and help to manage usage tracking and access control. Many AI services operate on a pay-per-use model, meaning that every request that is sent to the API is counted towards your usage. This is often tracked using tokens, which are used as a way of measuring the number of words in your request and the model's response. Therefore, protecting your API is



essential to avoid unauthorized access or unexpected costs. There are several ways to securely manage API keys without exposing them in your source code. One of the simplest methods is using environment variables, which store sensitive information outside of your codebase. Another option, especially useful in local development, is a secret manager in Visual Studio, which allows you to store secrets securely while keeping them out of source control. And for production scenarios, a more robust solution is Azure Key Vault, a widely used service for securely storing and retrieving secrets. Azure Key Vault provides tight security controls and ensures that sensitive data is only accessible to authorized application and services, both during development and in a live production environment. And no matter which approach you use, never hard-code API keys in your application. Securing them properly is crucial for both security and cost management.

Demo: Storing Keys Securely

In the final demo of this module, let's look at the options for working with keys in a secure way in our applications. So let's say that you want to store a secret like the API key in an ASP .NET Core Blazor application. Now your first instinct might be to place this in the `appsettings.json`, or perhaps the `appsettings.development.json`, since it is just another setting. But that is not a great idea. Storing an API key in plain text means that it could easily be checked into source control and exposed to anyone with access to the repository. Clearly that is not what we want. A much better approach is to use user secrets, which are designed for securely storing sensitive information during the development. And to set this up, I right-click on the project here and I go to Manage User Secrets, and this action creates a new secrets file on the local machine, which is stored outside of the project and won't be checked into source control in this file. I create in this case a section, `ModelSecrets`, and I stored the `ApiKey` along with its `SAMPLE_VALUE`. Now since this is a local file, it's safe and private and only accessible on my machine. If we take a look at the project file, you'll see that it now includes a reference to the secrets file, linking it to the project. Now, in order to read these secrets in the application, I add the necessary configuration code in my `Program.cs`. Now this setup here ensures that the secrets from `appsettings.json`, environment-specific app settings, and the user secrets are all read out. If there are duplicate keys, the last one added will be overwriting the previous values. And additionally, I include the `AddEnvironmentVariables` in the configuration setup. This is useful when deploying to environments like Azure App Services, where we can securely store the real API key. Now since the environment variables are added last, they'll take precedence over any previous values, ensuring that the correct key is used in production. Now, to work with these in a typed way, I'm using the options pattern here. I registered by binding the `ModelSecrets` section from the configuration to an instance of the `ModelSecrets` class. This class reflects the structure of the



secrets file, ensuring strongly typed access to the values. In my code, in my Blazor code that is, I inject the ModelSecrets instance using dependency injection, and it is done by using the IOptions in ModelSecrets interface. And once injected, I can easily access the ApiKey through its properties. When I run the application, the value is successfully read from the local secrets file and displayed here on the screen. So the important key takeaway here is that this value stays private. It is not exposed to anyone else, keeping the API key secured during development. This approach is simple, clean, and effective for managing sensitive information, ensuring that secrets remain secured and outside of source control.

Working with the Kernel and Prompts

Doing More with the Kernel

As soon as the kernel is really the most central part of Semantic Kernel, there's much more to learn about it, and we'll do exactly that in this module. Next, there are also quite a few ways to work with prompts, including handlebar syntax. We will look at that in this module, too. So let's dive in and broaden our knowledge about Semantic Kernel. And so we'll kick things off by diving deeper into the kernel itself, since that is where everything starts. Let's do a quick recap on what we have already learned about the kernel. We've already seen that the kernel is at the heart of Semantic Kernel. The kernel serves as a container that orchestrates interactions between AI models, plugins, and other things like memory. Now don't worry if I'm using terms here that aren't clear yet here, like memory. We'll come to these later. It acts as an abstraction layer over various AI services, allowing us to integrate with different models without tightly coupling our code to a specific service provider that increases the maintainability of the code we'll write in our apps. Next to managing the AI calls, the kernel also takes care of the execution flows, retries, and error handling, ensuring a smooth and robust interaction with AI models. Indeed, I don't think I mentioned that already, but behind the scenes, if an AI call fails, Semantic Kernel will also handle the retry for us. So the kernel has quite a few functionalities. I've created the list here, but there are definitely more that we'll come across throughout the remainder of this course. First, perhaps foremost, the kernel is responsible for selecting the most relevant AI service, ensuring that the right model or API is used based on the request. It can render prompts from templates, making interactions more dynamic and reusable. We haven't talked about prompts and definitely not about prompt templates. We will come to that later in this module. Once a prompt is prepared, the kernel invokes the AI service, sends the request, and if needed, routes it to plugins that extend the functionality. You've seen the letter already briefly in the previous module, but we'll look at that in more detail in the upcoming one. After receiving a response from the LLM, the



kernel passes the output and forwards it to the application for further use. Additionally, the kernel provides event hooks, and those allow us to add custom logic at different stages in the AI workflow. So as we can see, the kernel does quite a few things under the covers for us that we may not even realize it's actually doing. There are multiple ways to create the kernelling code. In the previous module, we've already seen this one, and I reckon this is the simplest way to create one. I'm starting by calling `kernel.CreateBuilder`, which provides a fluent API for configuring the kernel. Next, I'm using `AddOpenAIChatCompletion` to integrate an AI model for chat completions. Here we specify the `modelId`, which defines which AI model to use, and `apiKey`, which authenticates our request with OpenAI in this case. And as said, other methods exist to plug in other models, like Mistral or Google Gemini. We will see those in the demo. Finally, I call the `build` method, which creates a fully configured kernel instance which is ready to interact with AI services. Now, a more structured approach is also possible using dependency injection, and this code should look familiar if you've worked with dependency injection. Instead of directly calling the `kernel.CreateBuilder`, we first create a `builder` instance and configure its services. So in the snippet we see here, we add logging to enable debugging and attach an HTTP client, which can be useful for AI service calls. Then we register OpenAI chat completion, just like before, specifying the model and the API key. And finally, instead of immediately creating the kernel, we call the `builder.Build`, which ensures that all configured services are injected properly. I would suggest using this approach for larger applications where we want more flexibility, such as integrating multiple AI services or adding things like observability or caching.

Demo: Setting up the Kernel Using DI

So let's now head over to Visual Studio for a demo where we'll see the different ways to create the kernel in action. We're now in the `m2` solution folder for all the demos in this module, and in this demo we'll look at the methods in the `KernelSetup.cs` file. So in this first method, the `BuildKernelWithOpenAIChatCompletion` method, I'm in fact using the approach that I already used in the previous module. So we start by using the kernel using the `kernel.CreateBuilder`, and to enable OpenAI chat completion, we add the `AddOpenAIChatCompletion` method. This is also where we specify the model ID, which is passed into the method and the API key, which should be securely stored and retrieved. This approach is a straightforward and efficient way to set up the kernel. In the rest of this code, nothing new is really introduced. I have a prompt, and with the just created kernel, I execute that prompt passing in also some settings. Now next to this, let's say standard method, we can also use dependency injection, and that's what I'm doing here in this second method, the `AddingMoreServices` method. I'm basically extending the setup of Semantic Kernel by adding more services using dependency injection. I'm again initializing



in this line the kernel builder using the `kernel.CreateBuilder`, and on that kernel builder we get access to the `Services` collection, which we can use to register other services with the dependency injection system of .NET. That is what I'm using here next to register more services, like, for example, logging, which I add to `AddLogging`, passing in the configuration. Next, I'm also using the `AddHttpClient`, so both are regular calls to bring in other services into the `Services` collection. And finally, I configure again OpenAI chat completion using the `AddOpenAIChatCompletion` method. And then with the required services added, I use the `builder.Build` method to get access to the kernel. Note that every call to the build method will return a new instance of the kernel. In this final method, I'm using the service provider instead to create my kernel instance. We start by creating a new instance of `ServiceCollection` with this line here. First, I use the `AddHttpClient` again on that. Next, we register the kernel service itself, and it is done using `AddTransient` passing in `kernel`, meaning that a new kernel instance will be created each time it's requested. And inside the registration, we use a Lambda expression to define how the kernel should be created. Within this Lambda, we first retrieve the `IHttpClientFactory` using the `sp.GetRequiredService`, passing in the `IHttpClientFactory`. This allows us to generate an HTTP client instance. We then build a kernel using the `kernel.createBuilder` and configure it to use OpenAI's chat completion service using again the `AddOpenAiChatCompletion` as we did before. And to use the services, including the kernel, I built the service provider using `collection.BuildServiceProvider`. Note the use of the `await` using here, which makes sure that the resources are properly disposed when they are no longer needed. Finally, I then get the kernel instance using this `serviceProvider.GetRequiredService`, passing in the kernel type. And that kernel instance I'm then using to invoke the prompt. So what you've seen here are different ways to initialize the kernel instance in your application with and without explicit use of the dependency injection container in .NET.

Demo: Working with Different AI Services

As mentioned already a few times, it's very simple to use different AI services in combination with Semantic Kernel, but so far I haven't shown that yet. Let me do that in this demo. First, I'm going to show you how we can work with another service, and that's Mistral. The fact that Semantic Kernel abstracts away the differences of the various models becomes clear this way. To work with Mistral from Semantic Kernel, we first need to head over to the project file where we can see that I've added the `Microsoft.SemanticKernel.Connectors.MistralAI` package to show that Semantic Kernel knows about Mistral AI and its features. In the code in this method, I'm using `Mistral`. Semantic Kernel comes with a method that is `AddMistralChatCompletion`, which, as the name gives away, is aimed at Mistral to enable and use chat completion. And the good news for us is that the rest of the code is exactly the same because of, indeed, the fact that Semantic Kernel



abstracts the differences away. Running this, of course, works in the same way. We will get a response from the AI model. In this next method I'm going to go a step further, and in one go, I'm registering two AI services, OpenAI and Mistral. So at this point, both services are available for the kernel to use. And to control which service is selected at runtime, I'm registering a custom AI service selector using the `Services.AddSingleton` method here, and I'm injecting an instance of the `RandomAIServiceSelector`, which is a class I have defined over here. In here, one of the services is randomly picked. So this means that every time we send the prompt, the selector will randomly choose either OpenAI or Mistral. Of course, the logic here can be whatever you want it to be. I use a random value here just to show that. And the rest of the code should already be familiar. And if we run this now, we'll see that the service is selected and that the prompt is executed using the selected model. So the key takeaway here is that we can effectively register and use multiple AI services with Semantic Kernel. And since it is an abstraction layer, the way that we interact with the different services is identical.

Working with Chat History

The one important thing to remember about AI services is that they are stateless. Each and every time that we send a request to an AI model, it does not remember previous interactions. You can compare it to a web application and a web server, where, by default, once the response is sent to the client, all context about the request that was just handled will be removed. Each request is handled independently without context of previous conversations. However, when chatting using ChatGPT, it does continue a conversation, right? Indeed. But if you want to use this in our apps as well, you'll have to manually provide the context with each request. Adding chat history is the solution to this problem, and it is something that the application using the AI model must provide. Using chat history, we will provide the previous conversation messages and thus create more context, much more context, let's say, for the AI to work with, enabling it to provide an answer that is based on previous messages or previous interactions. So the chat history is therefore nothing more than indeed the list of previous messages in a chat, which can come from different sources. Of course, the user entered messages will be included. But also the response from the AI, so from the assistant, will typically be included, so that the AI can improve on them, for example. It has no way to remember its previous messages, so we'll have to send it along. Outputs from an external tool, so, for example, a plugin that provided the current weather in the location, and even system messages that help the AI behavior will all be included in the chat history. So this is what we see here. Typically things start with one or more system messages, so they're added to the chat history. Quite logical, if you ask me, since these need to be used by the AI for every message it'll produce. The user message, so perhaps the question will ask the AI, is also added into the history. When the AI service then generates its response, that, too, is included in the history, so that next interactions will



know about what the AI has previously answered. But the user adds a new question. The whole chat history is included in the request to the AI service so that it has all the context it can get to create a new relevant response.

Demo: Working with Chat History

Using the chat history is built into Semantic Kernel. So let's in this next demo take a look at using it in our sample. And the code for this demo is in the `UsingChatHistory.cs`, where I have again created a few methods which will show how we can work with history when having a conversation with an AI model using, of course, Semantic Kernel. As mentioned, models are stateless, and from applications such as ChatGPT, we may get the impression that the model knows what we are talking about. That is not the case. It is stateless, so that sort of fake conversation ID is actually because of the application that sits in front of the actual AI model. So that's exactly what we're going to recreate here. We'll need to send in a set of previous messages so that the model has the context to understand the conversation that we're trying to have. Semantic Kernel comes with this chat history type, which you see being used here. First, notice I'm this time using an OpenAI chat completion service, a bit of a shortcut to work with OpenAI chat from Semantic Kernel. Instead of creating the kernel and adding the chat completion service to it. I don't think I had shown that before, so this is the first time we encountered this approach here. But it works the same and it does the same thing. Next, I'm then creating a chat history instance as mentioned, which will be the key part here in making sure that the model can get in the previous messages and then create an appropriate response. We start by adding a system message using `chatHistory.AddSystemMessage`. This message sets up the behavior of the assistant. This is typically used to set the expectation and the context for the AI. Next, we add a user message using `chatHistory.AddUserMessage` over here. This is the query I want the model to answer. And with the chat history then prepared, we invoke the model by calling `chatService.GetChatMessageContentAsync`, passing in the chat history, and we could, in fact, here also pass any settings again. Finally, I'm also adding the assistant's response back into the chat history using this line here. This is important because if the conversation continues, the model will have access to its previous response this way, allowing it to maintain more context. In this next method here, which I'll run, since history is more visible there, I am doing more or less the same. This time I'm using the regular way to create the kernel again, and then I use the dependency injection behavior to give me access to the chat completion service. Then I create a loop. That is going to ask the first question, added to the chat history, and I'll show that. Again, the response is also added to the history. Now to show you the differences, I'm going to run a sample first without chat history and ask who was the 40th president of the United States. We'll get an answer and find out. So without the concept of history, ask, and what about the 42nd? We'll see that the model has lost. It is no longer aware



of the content, which was just a line before, but it's the app that knows about that, not the model. That is stateless. If we now run our current sample with the history enabled, you will see that when I do the exact same thing, we'll get a good answer. Also for the second question, since the conversation history contains enough context for the model to understand we are talking about US presidents, this is the history working for us. In this third method, which I suggest you take a look at your own pace, I'm combining the streaming responses with the chat history. If we run that, we'll see the same, but now in a streaming format again. We can also look at the entire message history. That's what I'm doing here in this method. I'm again creating a history and I'm passing already in a first message. I then perform the call to the AI model here and add a response to the chat history. And then with this loop I'm printing out the entire message history so we can see how the conversation has evolved.

Reducing the History

I've mentioned the word tokens already a few times now in this course. So let's take a moment to explain what that actually means. And you don't need to know all the technical details to use Semantic Kernel, but having a basic understanding about tokens can be useful. So tokens are the building blocks that large language models use to process language. But what exactly is a token? Simply put, a token can be a word, for want of a word, or even just a character, depending on how the model interprets it. When you provide input, the model breaks it down into these smaller chunks, allowing it to analyze and generate language more effectively. Think of it as the model simplifying language into meaningful pieces, really. So, for example, if you type Bethany's Pie Shop has the best apple pies, the model might split it into tokens like Beth any 's Pie Shop has the best apple pies. Notice that even a single word like Bethany's can be broken into multiple tokens. Now, you may be thinking, why is this important? Why does this matter? Well, most AI services, including OpenAI, charge you based on the number of tokens processed. So understanding how text is tokenized can help you manage the cost and also optimize the performance. Additionally, models have a maximum token limit, which affects how much information you can send in a single request. Of course, if you think back of what we just saw about including always the entire history, this can be a bit contradictory. On one hand, I'm saying we should include all past messages to create a context so that the AI knows what the conversation and the new request is all about. On the other hand, if the AI builds is based on the number of tokens to the length of the request, it will become very expensive. And indeed, that will happen if you keep adding messages to the history, which is then sent along with every request. Things will also slow down because the history becomes so long that the model will need to pass more and more messages, and you may even exceed the maximum length. To solve that problem, Semantic Kernel has the ability to shorten the history. Either a way to truncate the history, or, and this might be more clever, to



summarize it. If you want to reduce the length of the history, the Semantic Kernel comes with an interface called the `IChatHistoryReducer`, which we see here, and this interface allows us to create a custom implementation to reduce the chat history. Additionally, and we'll see this in the upcoming demo, Semantic Kernel also comes with two built-in ones, namely the `ChatHistoryTruncationReducer`, and the `ChatHistorySummarizationReducer` classes. The first one truncates the history to a set size and thus removes all older messages, and it is triggered when the chat history exceeds the limit. The second one, so the `ChatHistorySummarizationReducer`, truncates the history by summarizing a set of messages, removing them from the history, and then adding a single, the summarized version, to the history.

Demo: Reducing the History

In this next demo, I'm going to show you this reduction process in action in Visual Studio. This part of the code is already familiar. I'm manually creating a few messages, which I'm adding to the history. Notice here I'm creating an instance of the `ChatHistoryTruncationReducer`, and I'm setting the `targetCount` to 2. This means that as the chat progresses, the reducer will ensure that the chat history only keeps the two last messages, effectively minimizing the size while retaining, well, the most recent context. This is key for optimizing performance, and, as said, avoiding unnecessary context overload. In the loop here, I'm just adding messages one at a time, and then I call the `reducer.ReduceAsync`, passing in the history. As we already saw, this method then analyzes the current chat history, and if it exceeds the target count, it trims it down, keeping only the most recent messages. If that reduced history is not null, so truncation happens, we create a new chat history containing the reduced messages. This makes sure that the history remains concise and contains only the more essential context. Then invoke the model and add a response to the history, too. Let's run this and look at the behavior. As we can see while this runs, the conversation keeps running fine, even with the truncated history. And in this case, the prompts contained, looking at the results, enough context so that the AI model could still create a good response. Now, this still worked fine, but with the message history just being two messages long, it could be that we don't have enough context with just these last two messages. For that reason, it's actually better to use the `ChatHistorySummarizationReducer`, which adds a message to the history that summarizes the history. And therefore, that can lead to better results, given that the history isn't just thrown out, it is still kept in the form of a summarization message. If we now run this, I will put a break, we'll now see that the message containing the summarized history is added.

Crafting Prompts



Crafting good prompts is essential for guiding how the LLM should behave. A prompt serves as the input of the query that directs the model's response, influencing both the accuracy and relevance of the output. In this part, we will explore how to design effective prompts to get the best results from the AI models. And even with the help of Semantic Kernel, the prompt remains an important part of the whole equation. Before we dive into crafting effective prompts, let's first understand what a prompt actually is. At its core, a prompt is simply an instruction or input that we give to an AI model to get specific responses. It is how we ask the model to do something, whether it's answering a question, summarizing text, or generating an image. The key thing to remember is that the quality of the response depends heavily on how well the prompt is crafted. A well-structured prompt provides clarity, sets the right expectations, and ultimately leads to better results. In other words, a better prompt will result in a better response. So let's learn what we can do here. And let's start with an example of, let's say, varying levels of prompts. A bad prompt is often too vague or perhaps also too short. If you simply ask, "Create an article," the AI has no direction. It doesn't know what a topic should be, leading to an unpredictable or generic response. A good prompt adds some clarity, like "Create an article about AI." This gives the model a subject, but still leaves room for broad interpretation. To get the best results, we need to be more specific. A better prompt will be "Create an article about AI and focus on the benefits that AI brings to software development." That provides a clear scope and purpose. It's no wonder that this last one will give us what we're looking for. So when using prompts in our Semantic Kernel-based applications, it is crucial that we also always include clear, specific, and often longer prompts. There are a few techniques to learn in this area. First is zero-shot learning. It's a technique where the model is asked to complete the task without any earlier examples in the prompt. In this case, we're asking the model to determine the sentiment of a given text. The instructions clearly define the possible responses, positive, negative, or neutral. And I also specify that if the sentiment is unclear, the model should default to neutral rather than guessing. And since the model hasn't been given any examples, it must rely purely on its pre-trained knowledge to classify the sentiment. Hence, the name, zero-shot learning. This approach is useful when we need a quick response, but it can also lead to inconsistent results. Next to zero-shot is few-shot learning. This improves the accuracy by providing the model with examples before making a prediction. In this case, we're asking the model to categorize support requests into pre-defined categories. Now unlike what I did with zero-shot learning and all a model with no context, here I'm giving it three examples of user input, along with the correct classifications and that you can see in the highlighted part. Another way to improve the quality of the response is adding a persona. Instead of just asking the model to perform a task, we can define a role that influences how it communicates. In this example, we specify that the AI should act as a highly experienced software engineer with a passion for teaching. This persona helps ensure that the explanation is clear, engaging, and beginner-friendly. We also provide guidelines



on tone and style, instructing the AI to use a friendly language, everyday analogies, and step-by-step explanations. Finally, we also encourage the AI to include the C code sample when relevant, making the explanation more practical.

Demo: Working with Good and Bad Prompts

We've now seen how we can improve our prompts. Let's apply this in our examples and see how these different improvements can really help the quality of the response. I will go through several examples, and we'll start with this one here called `UsingBadPrompt`, which, as the name implies, is going to use, well, a pretty bad prompt. The prompt is "Give me a recipe." Now, this is a bad prompt because it is not specific. The model doesn't know what kind of recipe we're asking for. We're talking about a cake, a pie, a salad. The result will likely be vague, and it's entirely up to the model to guess what we want. This is a good example of how unspecified prompts can lead to generic or poor responses. Next, in the `UsingBasicPrompt`, I have improved things a bit by adding the type of recipe. So I said here "Give me a recipe for an apple pie." It is still lacking detail, though. The model now knows that we want an apple pie recipe, but it doesn't know how we want the information presented. Should the ingredients be listed separately? Should it use metric units? This prompt might result in a basic recipe, but it could miss details or not have the format we want. So while we're being a bit more specific, we can still improve. So in this `UsingStructuredPrompt`, I've added more structure to the prompt. I'm pretty specific what I want, so it's very likely that the response I'll get back is what I'm looking for. By specifying the format and measurement units, we guide the model to produce a more useful response. It is all about the expectations here. In this example, I have taken things a bit further and I'm introducing a persona. I'm basically saying what role the AI model should adopt when producing the response, and this often leads to more detailed and, well, let's say professional responses. In this method here, using the single-shot front, I'm introducing the concept of single-shot learning. I'm giving the model one example so it can align the output with that. So the model will learn from that single example and mirror the style in its response. This can significantly improve the consistency and the formatting. In the `BuildKernelWithFewShotPrompt`, I'm even extending on this to get an even more aligned result. By seeing multiple examples, the model will better understand the desired format and tone. This helps it to produce responses that are consistent and detailed. This typically leads to the best results, but also takes the most time to create, of course. So while indeed, it takes a bit more time to create a good prompt, it is definitely in your own benefit to spend a little bit more time on it to get good answers.

Using Prompt Templates



So far we've just seen templates as being a simple string, but there's also the option to work with Semantic Kernel prompt templates, an easy way to define and compose AI functions using just plain text. Prompt templates allow us to create dynamic reusable prompts by incorporating elements like variables, function calls, and function parameters. Variables let us insert dynamic content into a prompt. For example, instead of hard-coding a user's name, we can use the `$username` between braces and fill it in at runtime. It's also possible to from within a prompt call an external function and include the result of that inside the final prompt, and we can also include parameters for these functions. All these work inside the prompt by using a set of double curly braces. Semantic Kernel will pause the template and execute a logic that's included inside the template. So we can indeed create more dynamic prompts this way, and that gives us some added flexibility. Here are some templates and examples. The first line, "Hello `$name`, welcome to Pluralsight!" uses a variable placeholder, `$name`, allowing us to personalize the message with the user's name dynamically. The next one, "The weather today is `weather.getForecast`." That shows how we can call a function inside a prompt to fetch real-time information. In this case, a weather forecast. The final line, "The weather in `$city` is `weather.getForecast $city`" takes it further by using a variable, `$city`, to make the function call location specific.

Demo: Using Prompt Templates

Time to see how we can use prompt templates in action in the next demo. I'm going to be leveraging prompt templates to build a prompt that contains customer preferences and thus can be more tailored to what the user wants. Remember, prompt templates allow us to build flexible prompts that can change based on input values. Let's take a look at the code together. First, of course, I'm setting up the kernel. Nothing new here. But next, I'm defining the prompt with the prompt templating syntax of Semantic Kernel. The fact that I'm using the three quotes here is because I'm using a multi-line string syntax in C#. I was doing that already in the previous example, by the way, as well. The prompt is a good prompt, setting as the first thing in the scene for what we'll want to do, and that is recommending a pie based on the customer preference. And the template includes two variables here, namely the customer name and the customer preference. These variables are placeholders that will be dynamically replaced with the actual input values when the prompt is processed. The rest of the prompt is instructing the model how it should create the output. Now after defining the template, we set up input variables using the `KernelArguments` object. This is where we provide real values for the placeholders defined in the template. Now here I'm hard-coding these, but these could, of course, come from anywhere, including a database. Using the templating syntax, I can use the same structure and template for different customers by just changing the parameter values. Then I create a semantic function using the `kernel.CreateFunctionFromPrompt`, passing in the template. This step tells



Semantic Kernel to pause the template and prepare it for execution. And using `kernel.InvokeAsync`, I then execute the function, and basically this is where Semantic Kernel will again take it over from us. It sees that the template has variables declared, and it will dynamically replace these. And finally, of course, we'll also send the prompt to the AI model. What makes this approach so powerful is its flexibility and simplicity. By using variables within the prompt, we can dynamically customize the output without changing the underlying template. Next to this approach, we can also work with the prompt template factory instead. This approach comes with some additional advantages. More on that in just a second. Let's first take a look at the code. It is definitely a bit more complex to set up. First, you can see that the prompt stays the same as before. I'm also setting up arguments like we did in the previous method. Next, here is where the process differs. Instead of directly creating the function from the prompt, I'm using here the `KernelPromptTemplateFactory`. This factory approach is very useful when you need to manage multiple templates or more complex configurations. We start by creating a new `promptTemplateConfig`. That's what we see here. This configuration object allows us to define details about the prompt in a structured way. The first property, the `Template`, is set to the prompt text we defined earlier. The `TemplateFormat` is set to `semantic-kernel` to ensure that the factory understands the template format, and the `Name` I'm giving is the `PieRecommendationPrompt` to identify uniquely each template. I'm using this line here. I'm using the factory to create the `promptTemplate`, and it's using the `promptTemplateConfig` we just created. Next, as a bit of a sidestep here, I'm rendering the prompt using this line here. This now allows me to see the rendered prompt before it's actually sent to the AI model. And using `kernel.CreateFunctionFromPrompt` passing another config and the factory, we create the function, and next it's being invoked by Semantic Kernel again to send to the AI model. Let's now run this to verify that this works correctly, and we'll also in one go see the rendered prompt. So here's the prompt indeed, and here is the response from the model. I still owe you the advantages of using this second approach here. And this brings in added scalability and flexibility, making it easier to manage multiple templates and configurations. And also the ability to render the prompt before invoking it is easier for debugging purposes and also allows us to ensure that the prompt is correctly structured.

Using HandleBars Template Syntax

Next to regular prompt templates and the corresponding syntax, there's another type that we can use called the HandleBar template syntax. HandleBar is a simple templating engine, often used for HTML, but it can also be used to generate other types of text. HandleBars allows us to mix static text with dynamic expressions. It works by embedding HandleBars expressions directly into the text, making it easy to replace placeholders with actual values at runtime. And to use it, we need to bring in an extra NuGet package to enable support



for HandleBar's prompt templates in Semantic Kernel, and I'll show you this in the demo. Whereas the regular template syntax is mostly about variables and function calls, HandleBars can do a bit more. It also supports logic-based structures, like loops and conditionals, making prompts more dynamic and adaptable. Of course, variables are supported, allowing us to insert dynamic values. Loops are also supported, enabling us to iterate over items, and also conditionals are supported.

Demo: Using HandleBars Syntax to Create a Prompt for Semantic Kernel

I just head straight back to Visual Studio. I will see the HandleBars syntax in action to work with prompts. And we already saw the regular syntax in the previous demo, but as we'll see, the HandleBar's syntax can go a bit further in that it supports things like loops. First, in order to use HandleBars templates, a new package is required to be added, and that's this one here, the `Microsoft.SemanticKernel.PromptTemplates.Handlebars` package, which, as you can see, is already added in the project file. The code is here in the `HandleBarsTemplates.cs` file. We have here a HandleBars-based prompt template, which is defined again as a multi-line string. This prompt is designed to simulate a conversation with an AI assistant at Bethany's Pie Shop again, where again the assistant recommends pies based on a customer's preferences. So that is this part here. Next, in this part here, I'm using the customer details using HandleBars syntax to work with variables. I am using the dot notation here to access the properties, the nested properties, let's say. This is one of the strengths of HandleBars as well. It allows for clear and structured access to data within objects. Next, in this part, I'm instructing again how the model should structure its response. Finally, the template includes a loop using the HandleBars `each` helper to loop over the history collection. This allows dynamic rendering of multiple messages from the conversation history. So basically what this will do is looking in the history argument and including previous interactions in the prompt. And each message in the history is then rendered with its role and corresponding content. Next, I'm defining the input arguments using the `KernelArguments` object. This is where we provide the data that will replace the variables in the template. The customer has a first name and last name, the preference is included, and so is the history as an array containing the previous messages. Next, I'm using the HandleBars `promptTemplate` factory, which is similar to what we used in the previous demo as well. Also similar is the rest of the code here. I am using the `promptTemplateConfig` and the `promptTemplateFactory`, so I won't be explaining that here again. So let's run this to validate that everything works fine. And there we go. We see the response, including the data coming out of the loop of the history and messages. So to summarize this part on HandleBars syntax in Semantic Kernel, it offers greater flexibility and also control, especially when working with complex data structures. And it can also help us when working with



nested objects, like customer details, and supports loops like what I've shown here to add the conversation history dynamically.

Extending Semantic Kernel Using Plugins

Understanding Plugins and Functions

Semantic Kernel is powerful on its own, but its real strength comes from extensibility. In this module, we will explore how to extend Semantic Kernel using plugins, allowing us to integrate custom functionality into our applications that gives the AI service extra data to come up with a better response. In this module, we'll look at plugins in much detail. We have touched on plugins and functions already briefly earlier in this course. But let us now learn, well, a lot more about them. And let's start here with a quick recap. We already know that AI models are really powerful, but they can't know everything simply because there's a lot of information not accessible for LLMs to get trained on. They are trained on publicly available data and typically on all that available data until a cutoff date. But when it comes to real-time data, private systems, or business, so internal data, they don't have that all available to them. That is where plugins come in. By exposing existing APIs and services to an AI model, we can give it access to local or external data, enabling it to interact with life systems, databases, or third-party services. If you have used ChatGPT function calling, you've seen how AI can fetch live data, book reservations, or retrieve up-to-date stock prices. Well, Semantic Kernel plug-ins work in a similar way, as we'll see. So as said, the way that a function calling works for most AI services is very, very similar. And let's make sure we understand that first using this graphical representation. The process begins when your application sends a request to the AI, including the prompt and function definitions that the model can invoke. These functions, essentially just C# methods, allow the model to call back into your application when needed for getting extra information. It's important to provide clear descriptions of what each function does so that the model can use them effectively. Once the model receives the request, it analyzes the input and determines whether it can respond directly, or if calling one or more functions would improve the answer. Then if function calls are needed, the AI service responds by specifying which function should be executed. And it also supplies the necessary arguments. Your application then executes that function locally, which might involve getting real-time data, interacting with the external services, or performing a specific action. After that execution, your application sends a follow-up request back to the AI model, including both the functions result and the original prompt. And then finally, the model processes this new data and generates a final response, incorporating the function's output into its reply, ensuring a more accurate and context-aware answer. So this is how function calling works, and plugins and functions in Semantic Kernel do exactly



this. When using OpenAI's SDK directly, writing these functions is quite a lot of work. With Semantic Kernel, the process to use these becomes really easy. So plugins can expose pretty much all type of functionality for an AI to use, so to call back into, let's say. The most typical example is giving the AI model access to up-to-date information, such as weather or stock information, typically something that the model can't know because it's real time. Another typical example is making an appointment in a user's agenda. This is not really information, it's a task, really, that is by default not possible for an AI to perform, since my agenda is local, perhaps on an internal mail server. But by exposing this as a function, it becomes possible for an AI to instruct to create an appointment on a given date and time. Another example is a shopping assistant, which can use plugins to compare prices across different online stores, helping users to find the best deals. Again, that is real-time data. So by default, an AI service can't do this. In general, AI's can't access internal company data, but with plugins, it can retrieve and interact with private databases, ERP systems, or business applications. So this list is practically limitless. Function calling is how OpenAI enables this, but as said, that is quite a lot of work. When using Semantic Kernel, it becomes really easy to expose local functionality to an AI service through the use of plugins. Plugins are really just classes which contain one or more functions exposed to the AI service so that the AI model can invoke these when it needs to. One of the key advantages of using Semantic Kernel plug-ins is that their orchestration is mostly automatic. The AI model decides when a function is needed, what parameters to use, and then calls it without manual intervention. These plugins, so these classes, are often referred to as native plugins, since they're written in the same native language and framework. So in our case, internet and C#. For an AI model to use plugins effectively, it needs to understand how each function works. This means that every function must include a clear description of its functionality, along with the expected input and output. And by adding this information, we enable the model to determine when a function can be used and how to call it appropriately. Here's a snippet containing a simple plugin in Semantic Kernel. We have a TimePlugin class that defines a single function. As mentioned, a plugin is just a class with one or more kernel functions. And in this case, our function here will allow the AI model to retrieve the current date and time in UTC format. And in order to be exposed to the AI model, we must add the KernelFunction attribute. This marks the method as a callable function for the AI. Then the Description attribute here provides a clear explanation of what this function does. This is important because as said, the AI uses this description to decide when to invoke the function. And then, finally, the method itself will just return the actual UTC time. And just adding this plugin doesn't do anything. We still need to register it with Semantic Kernel. There are several ways of doing this, but most often you'll use the input plugin from type, which you see here, passing in the type of the plugin. Its kernel function will now become available to the AI model, which can then invoke them when it deems it necessary. Now, we don't manually invoke this plugin. That's important. It is not so that we will call this method ourselves somewhere in the code. We just



registered it, and Semantic Kernel and the AI service will do the rest. However, we do need to add one more thing here, and that is defining in our case the OpenAI prompt execution settings and setting the ToolCallBehavior to `AutoInvokeKernelFunctions`. This setting tells the AI that if a plugin function is needed, Semantic Kernel will handle the execution automatically. No manual intervention is required. And finally, we then call the `GetMessageContentAsync`, passing in the chat history, the settings, and the kernel. Invoking the plugin and the back-and-forth we saw a couple of slides ago happens entirely behind the scenes and is orchestrated by Semantic Kernel. Now, while plugins in Semantic Kernel are powerful, there are some common pitfalls to keep in mind when integrating them into your AI applications. First, remember that plugins are serialized when communicating with the LLM. Now remember the story about tokens and the length of the prompt? Well, indeed, plugins are serialized and are sent back as part of the message. So if you have a lot of functions, a lot of the bandwidth will go to that serialized list of functions you have. That will have an effect on the length and thus also on the number of tokens being used. In general, it is advised not to have more than 10 to 20 functions, not only because of the used tokens, but also that it might become harder for the LLM to pick the right function. Also, while you can and will use complex types as parameters, it's advised not to make them too complex. And finally, it can be useful to make use of function advertising. Function advertising is the process of providing functions to an AI model for further calling and invocation, and by default, all available functions will travel along. But you can change this so that only a smaller subset is sent along.

Demo: Using Plugins and Functions

Now it is time for quite some demos on using plugins in Semantic Kernel. We'll start here in this demo with learning more about plugins and functions. All demos can be found in the `m2` solution folder, and most things we'll do in this modular part of the `ExtendingWithPlugins` project. Again, as before, the references to the Semantic Kernel packages have already been added in the project file together with quite a few others that we will use later in this module. Other methods for this demo can be found in the `SimplePlugins.cs` file. Okay, so plugins. We have already seen that plugins in Semantic Kernel allow you to extend the AI's capabilities by registering custom functions that the model can then automatically invoke when it determines those functions are relevant to the conversation. In this method, I'm going to use a simple plugin, the `WeatherPlugin`. On the kernel, I'm using the `kernel.ImportPluginFromType`, passing in the `WeatherPlugin` type to register through this class with the kernel. This is all that is needed from our side to make the functions inside the plugin available for the AI to use. Doing this makes Semantic Kernel understand that the functions within this plugin are callable in the conversation with the AI model. We don't need to manually go and invoke these. Semantic Kernel will do that for us. And



before we look at how the conversation will happen, let's look at that plugin. That is just a class with two methods, two functions, if you will. In this case, both are going to give a sort of random weather forecast. Note that we saw when we looked at plugins indeed that these functions have the `KernelFunction` attribute applied onto them. This is what lets `SemanticKernel` know that these methods can be used in a conversation with the model. The first method here accepts just the location and returns a string representation of the weather. The logic is pretty straightforward, but the method also has the `Description` attribute, which lets `Semantic Kernel`, and there's also the AI model understand what this extra functionality can do for it. The same goes for the parameter, which also has an attribute applied onto it. This second method is very similar, but also accepts a `DateTime` parameter, which is also described. So the difference between these two methods, which are also reflected in the descriptions, is that the first one will be turned by the forecast for the current day and the second one for a given date. So key takeaway here is that using the `KernelFunction` attribute and the `Description` annotations will give semantic kernel all the information it needs to automatically determine when and how these functions should be invoked during a conversation. It is indeed `SemanticKernel` that will do a lot of the heavy lifting here. Back in the main code of the method, you see that I'm using a `ChatHistory` again, as I want to create a real chat experience. Next, we configure the execution settings using an `OpenAIPromptExecutionSettings` object. That contains a setting, the `ToolCallBehavior.AutoInvokeKernelFunctions`, which tells the kernel that it should automatically invoke registered functions like the ones in the random plugin. Whenever the AI model decides it is necessary to do this. In the loop, I then call the AI model using this line here, using again the `chatCompletionService`, and I'm using this time the `ChatMessageContentAsync`, passing in the history, the settings, and the kernel. It is now `Semantic Kernel`'s role to help the AI model. If the model detects that the user's question could better be answered by invoking one of the plugin functions, like requesting the weather forecast, it will automatically call that function, retrieve the result, and include it in its response. This all happens behind the scenes. No manual intervention required. Now to test things, I'm going to put a breakpoint in my two methods, so we'll see when each is invoked, which is triggered by the model and `Semantic Kernel`. So time to run this and ask for a weather forecast. So let's ask first, without specifying a day, what the weather is like here. And we'll see that after a few seconds, the function is called. So the model has sent back it would like to invoke the function part of a plugin. That we can now let execute. And a new call, including the response with the weather information, is sent back to the AI model, which will return and produce a nice response based on that. If I ask for the weather for another date, the other function is invoked. Indeed, that breakpoint is now hit. So that's just a simple example with one single plugin. Now we can add multiple plugins, and that's what I'm doing here. I've added two more plugins. The agenda plugin has two functions defined, which are exposed to the AI. This one here will set in a fake agenda I create over here and which initializes each day as a working day, the given day to a vacation day, and the second one



can be used to check if a given day is a working day. The third plugin is used to get, again, the current date and time. All these plugins are now included in what the application exposes to the AI model. The model is, however, very clever and can combine this with a lot of manual work. And I'll put some breakpoints again to see this in action. You'll see that it'll call if it'll rain according to our fake forecast, multiple plugins. First, it will check tomorrow's date. Then it goes to a random plugin and if that returns a little rain, automatically the agenda plugin is called. So this whole interaction is done for us because of the interaction between Semantic Kernel and the AI model. Now one more thing before we head back to the sites. In these methods we've used the input plugin from type, passing in the type. This we did straight on to the kernel. Other options exist, and I have them in this method here. I'm setting up the KernelBuilder as we did before, and I then use the `AddFromType`, passing in the type. The main benefit of using this approach rather than the first one is that it's now done in the initialization phase, which makes it so that it's easier to keep track of which plugins are registered. Another option is doing what I have here, namely creating an instance of the plugin and then using `AddFromObject` instead. This is useful if the plugin needs to be initialized with custom parameters or configuration values that might be required for its logic. Now the result of using any of these options to register a plugin is the same, and the result is what we saw happening earlier with our plugins being invoked automatically.

Demo: Using a Function with a Complex Type Parameter and Return Type

Most functions so far had simple types as parameters. When using a complex type, we need to do some more work for the AI service to understand what a plugin expects. The key difference in this demo is how the `WeatherPluginComplexTypes` handles input data. I'm accepting a weather request here in this second method. This approach is ideal when the input data needs to be structured or involves multiple properties. The method works mostly the same, but take a look at the `WeatherRequest` type. I've added a `Description` attribute on the different, well, let's say non-obvious properties, so that the model knows what needs to be sent along. Again, Semantic Kernel is key here. It'll generate a schema and pass that on to the AI service. Registering the plugin is identical to what we used before. All the different options that we saw in the previous demo can also be used with the plugin with a complex type parameter. If you run this, we'll see the exact same behavior, but know that behind the scenes, Semantic Kernel is working and saving us from having to do manual work, converting from and to this complex type.

Demo: Working with Function Advertising and Choice Behaviors



I mentioned that we can change that all functions will be made available to the AI service. Let us learn how we can change this using function advertising and choice behaviors. This is useful to influence the AI's decision-making process. We can give it full flexibility. We can guide each choice, or we can enforce strict invocation rules. The quality of the function choice behaviors is really centered around function advertising, which is a process to make certain functions available to the AI model. And by adjusting the function choice behavior, we can define exactly which functions the AI sees and how it's also expected to use them. There are basically three options, which are shown in this demo. The first one is the full flexibility option where we advertise all functions to the AI model, which then will give it freedom to use which functions, if any, it wants to. This is what we see in the `AddingAllFunctions` method. Most of the code will already be familiar, so I won't be spending time looking at this, so the focus is here on the settings. So by default, the auto behavior advertises all available functions from the registered plugins, and the AI can select, as said, whichever it wants to use. Also, it is not forced to use a function, either. The AI can come up with an answer itself, if it can do so. For example, if a user would ask what is the weather like in London today, the AI might decide to invoke the `GetWeatherForecastForLocation` function. However, if the question is less direct, like what is the weather like in Europe, the AI could choose to provide a general response without invoking any function. This approach is ideal when we trust the AI to make intelligent decisions about when to use external functions. It keeps the interaction, well, flexible. In this second option, we're advertising specific functions, but the AI can still decide which ones to use. Notice the change. After the creation of the kernel, I'm going to get one or more functions specifically. Then here I'm advertising in the settings that only this function is available. So now the AI has only access to this function, but can still decide whether and when it wants to use this particular function. Basically what we're doing here is guiding the AI's attention to its specific functions while still allowing it to determine when to use them. It is useful when we want to influence the AI's behavior without strictly enforcing it. Finally, here's the third option. I'm using a strict approach here where we enforce the AI to use a function to create a response. I have again retrieved a specific function, but now in the settings, I then enforce its use. If we now ask what is the weather in London, the AI cannot rely on own knowledge. It has to use this function. This is useful for when accuracy is really critical, and we must be sure that the creative response is based on the result of executing a function.

Working with Built-in Plugins

We have just built a number of plugins, but Microsoft also complemented Semantic Kernel with a set of built-in plugins. Now I did make a selection out of the available plugins here. The first one is the time plugin. Can you guess what it's going to be used for? Well, it does the same as the one we have already



looked at before, namely giving the AI model access to the current date and time. Next is the HTTP plugin, which lets the AI make web requests to external APIs, fetching live data or interacting with the web services dynamically. There's also the FileIO plugin, used for reading and writing files. This plugin helps manage stored content, making it useful for applications dealing with local or cloud-based file systems. There's also a text plugin, which provides utilities for manipulating and processing text, such as formatting, extracting information, or modifying content.

Demo: Using Built-in Plugins

In the next demo, I'm going to show you how we can use one of these built-in plugins. So we'll start with this method here in the `BuiltInPlugins.cs` file, which contains the sample for this demo. And let's start with the conversation summary plugin sample. As the name is given away, this plugin can be used to summarize a detailed chat transcript. The plugin simplifies the process of generating concise summaries from longer text inputs, which is especially useful in applications like chatbots. Here in the code I'm inputting on this line here this plugin, just in the same way as we did with the plugin we created ourselves. The input that we use is the generated transcript that you see here, and it's a transcript of a chat between friends to discuss weekend plans. So it can definitely be summarized, and that is the job for the plugin. We can use this plugin like the ones we've used already, so by registering it, and to test the plugin, we can invoke it manually, and actually that's what I'm going to do here. We haven't done that yet. I'm going to manually invoke a plugin and I'm going to invoke the `SummarizeConversation` function on the plugin, passing in the transcript, and so this line will return as a summary. Let's try that out. And yes, indeed, the conversation is nicely summarized. In this second method, I'm using a built-in version of a plugin that we already saw, and that's the `TimePlugin`. You can indeed see here that this is a plug-in that also comes with Semantic Kernel. We could input it and use it instead of the one that we have written and used before. In the code you see here, I'm doing the exact same thing. I'm manually going to invoke the plugin. And, in fact, I'm going to call the `Now` function on it. If we run this, we indeed see that this function is returning us the current date and time. And as we saw, there are other plugins built in available, but the way they can be used is always the same.

Adding File-based Prompt Functions

So far, we had our prompts always directly in the code. Turns out we can also place them in separate files to create what is known as a file-based prompt function. Let's look at these next. In Semantic Kernel, we can define prompt functions not just in code, like we've done so far, but also as file-based prompt



functions, also referred to as prompt plugins. These functions are stored in files, typically two files per function, organized in a separate folder. Now I'll show you the structure in just a minute in Visual Studio. One file defines the prompt template, while the other provides configuration details for the function. The Semantic Kernel automatically serializes these file-based prompt functions, ensuring that they can be seamlessly used within our AI workflow. As said, two files are involved to create these. First, we have the `skprompt.txt`. This file contains the actual prompt template that the model will use. It can include variables that get dynamically replaced at runtime, allowing for more flexible and reusable prompts. We've seen the replacement of placeholders already before. It works in a similar way. Secondly, we have the `config.json`. This file provides important metadata about the prompt function. It defines the type of prompt, the description, execution settings, and any variables required for processing. Now you may be wondering, what is the use of this approach, rather than placing the prompt directly in the code like you've done before? Well, there are definitely some benefits. First, they're easier to maintain. By storing prompts in separate files, we can modify them without changing the underlying code. Secondly, they promote a sort of separation of concerns by keeping prompts distinct from application logic, and that allows for cleaner and more modular development. Another advantage is that they can enhance collaboration. Teams can work on prompt engineering separately from the coding part. That makes it easier to iterate and refine prompts without disrupting the core application. And finally, file-based prompts are shareable. They can easily be reused across different projects or shared across teams, and that increases consistency and reduces duplication. Here is a sample configuration file. This one defines the execution settings for a Semantic Kernel prompt that provides special pie recipes. The schema field ensures compatibility with Semantic Kernel's configuration structure. The description field outlines the function of this prompt, that it is to generate recipes for baking unique pies. Within the execution_settings section, the default configuration specifies that the AI model should generate the responses with the maximum total limit of 250, which balances between response length and efficiency. The temperature is set to 0.7, which allows for more creative and varied recipe suggestions.

Demo: Using File-based Prompt Plugins

Time to see this in action. In this demo, we'll see how we can work with file-based prompt plug-ins. The code I'm using in this demo can be found in the `FileBasedPrompts.cs`, which contains two, well, rather simple methods. At first, it should strike you that there is no prompt in the code this time. Indeed, that now is in a different file since we're looking at these file-based prompts. Through the use of this line here where I'm using the kernel, and on the kernel I'm invoking the `CreatePluginFromPromptDirectory`, I'm instructing Semantic Kernel to go and search the Prompts directory, so that is this one, and it will automatically import



any prompt templates and configurations it finds. This approach allows us to keep prompts organized and easily manageable, so separating prompt logic from the core application code. So if we head over to that Prompts directory, we can see that we have a subfolder called PieRecipes, which contains two files, two important files, the config.json and the skprompt.txt. These two files will work together to define how the prompt is processed and executed. As mentioned in the slides, the configuration defines the settings for how the prompt should be executed. This schema is a versioning key. The description, well, that's simply a description of what the prompt is designed to do, and that is generating recipes here. And the execution_settings contains how the model should execute the prompt, including things like the temperature. Every time the prompt is invoked, these settings are being used, and we can now easily just change a .txt file containing things like these settings, rather than changing the configuration code inside of our application's code. The skprompt.txt contains the actual prompt that will be sent to the model. This prompt is structured as a user message, clearly instructing the AI to provide an original pie recipe with detailed preparation stamps and an indication that the metric system should be used. Again, same advantage, we can change the prompt without changing our application's source code. So with the kernel configured and the prompt files loaded, invoking the prompt becomes very straightforward, and that's what I'm doing here using this code, which we have already seen, and that is the kernel.InvokeAsync. I'm referencing here the PieRecipes using its full name, so when the kernel automatically matches this to the corresponding skprompt.txt and config.json files. If we run this, we'll see a simple recipe that is being shown. In a second example, which is very similar, I'm now bringing in a parameter. Most of the code is, of course, identical. In the prompt, you can see that the template syntax is being used here once again to bring in the value of a parameter. So that is this today here. And just like we did before, we can pass kernel arguments, which is what I'm doing here. I'm defining that the today argument should be passed in, and its value is also provided. If you run this now, we'll see that we'll get the recipe based on the season the current date is actually in. So to conclude, we can see that by using these file-based prompts, we can get some benefits compared to always putting them directly in the code. Things become, again, a bit more maintainable, a bit more flexible, since the prompts, as well as the settings, are now in easy to edit files.

Using RAG with Functions

In this next section, let's take a look at using functions for RAG-based scenarios. Now before we can explore this, I think I need to give you some more explanation on the concept of RAG. Retrieval-augmented generation, or RAG for short, is a technique to enhance AI-generated responses by bringing in relevant external data. Instead of relying only on the models built in training data, RAG allows AI systems to retrieve additional information from sources like



databases, APIs, or document repositories. This ensures that responses are more accurate, well-grounded in real data, and contextually relevant. So, in essence, we will somehow get extra data. We share that with the AI so that it can create a better, more grounded response. If we look carefully at the words in retrieval-augmented generation, retrieval is important. What we need to do here is, like I said, fetching data somewhere. Now if we think about it, we used plug-ins already to get extra data, and we'll see indeed that using functions or plugins is a way to do RAG. Through the use of this extra data, we enhance the generation side of things. Just like what we have already done using plugins, we will provide the AI model with extra data so that it can enhance its generated response. RAG will help us to steer the AI models so that the generated responses are better, like I said, grounded in real data that perhaps the AI doesn't know yet or simply cannot know because it's internal data. It will definitely help with avoiding hallucination, a problem that AI models have been known for. Where would RAG be useful? Well, the answer is in a lot of scenarios. A first typical example is customer support, where AI-powered chatbots and virtual assistants can pull relevant information from knowledge bases, FAQs, and perhaps past interactions to provide precise and helpful answers to customer inquiries. Another relevant area is research assistance, where RAG enhances the ability of AI to fetch the latest studies, papers, or technical documentation, allowing researchers and analysts to access up-to-date insights quickly. Finally, RAG is also being applied in healthcare, where accurate and timely information retrieval is definitely critical. Now it's one thing that we know that we should pass extra data to the AI model. We can use a plug-in. That seems obvious. With plugins being invoked automatically, we have a good way to pass extra data to the model to generate a response. But where does that data then come from? Well, the database perhaps? Indeed. Technically, in the context of AI, we refer to two types of search. The first one is search like we've known and used it for years, classic search, and it simply works by filtering items based on exact matches. This is the type of search typically used in traditional databases where queries are matched against indexed keywords or other structured data. However, classic search lacks any understanding of context, meaning it may not always return the most relevant results if the wording of the query does not precisely match the stored data. So it works great when we want to find a customer by ID, but it won't find related data. If the user searches for phone, for example, it's own telephone won't be found. That is different with semantic search. This typically uses vector databases to understand the context and intent behind the query rather than relying on the exact keyword matches. By using embeddings and similarity measurements, semantic search can retrieve information that is, let's say conceptually related to the user's request, even if the exact words do not appear in the database. So that's really cool. Choosing between semantic and classic search really depends on the query type. Semantic search is best for natural language queries and knowledge bases or customer support, while classic search really excels in exact matches for precise results. Often you'll need to combine both approaches. For example, an e-commerce chatbot can use



semantic search to understand queries and classic search to filter products by price or by availability, for example. So it should be clear that RAG is often combined with functions and plugins. These are used to dynamically retrieve and provide additional data to the model. Functions allow AI systems to pull in relevant information from external sources in real time. In most cases, retrieval is dynamic, meaning that the AI fetches data as needed based on the user's request. However, in some scenarios, static or pre-fetched data retrieval may be used where all relevant information is sent to the model upfront.

Demo: Using a Function for RAG

So time to head back to Visual Studio, and we'll see a simple RAG-based solution using a plugin. So what I'm going to show you here is how Semantic Kernel enables retrieval-augmented generation, or RAG, by allowing AI agents to call functions and retrieve external data in real time and thus give us an answer grounded in good real data. The data in our case is living in a local database on the system, so it's definitely not accessible by default for an AI to be trained on. The setup in this example is built around a common business scenario. We want to provide conversational access to sales data. So imagine a user interacting with a chatbot and asking how much did we sell last week or what are our top-selling pies in April. Now, while a language model can generate an answer to that question, it doesn't know our actual sales data. This is where RAG and also this demo comes in. Let's look at the code I've created for this. I, of course, create my kernel as we've done numerous times already, and next time I'm holding a plugin, that is the OrderPlugin. And this plugin will act as a bridge to live order data through functions that can be called by the AI when needed. Here's the code for the plugin. I have two methods in here to work with the orders. One will get all the orders for a given period, so between the start date and an end date. It's as we've seen already, marked as a KernelFunction, and it also comes with the Description so that the AI models know what functionality this method can offer. In the method, I use an OrderDataService to get the specific orders. That OrderDataService, what we can see here, so instead of going to a real database, I'm going to create some random orders here and return those. This second method of the plugin class, which is the GetOrderDetails method, gets detail of a specific order based on the order ID, and it will also use the OrderDataService to do this. Back in the main code to support function invocation, I'm configuring the prompt execution settings to enable automatic function calls using these AutoInvokeKernelFunctions. All methods of the plugin are now automatically advertised, allowing also the model to use them when it thinks it needs to do so. In other words, the AI can say, oh, this looks like a question about sales totals, and it can then call the function OrderPlugin to get the answer. In the rest of the code, we have a simple chat loop again allowing us to ask questions and adding them to the chat history. In here, the model will invoke the OrderPlugin functionality if needed, and the data retrieved by the



plugin function will be included, so retrieved and then added. And based on that extra data, the model can then use that to augment its answer to be reliable. So this is really a good example of dynamic data retrieval. The AI doesn't know ahead of time what the user will ask, and we're not preloading the model with possible order data. Instead, the model retrieves only what it needs when it needs it. So let's run this and ask a question. I'm going to ask you to compare the sales of January 2024 with 2025. And after a while, we'll get an answer, clearly augmented with our own data. We can also ask it to create an analysis of our sales of these months. Since we provide it in our function with lots of relevant data, the sales analysis we're getting back is quite nice, as we can see here. We can also ask it for the status of a specific order with a given order ID. In that case, we get back the information on the order. The plugin provides classic search functionality in this case, filtering by exact parameters, like date or product name, which is ideal for precise structured business data. So what we saw here was clearly a good example of using RAG to enhance, let's say augment, the response and the data provided came in through the use of a function.

Adding Filters for User Consent

So far we've seen that our functions just run, but it is possible to combine these with filters, and let's take a look at this next. Filters in Semantic Kernel play an important role in controlling function execution, and thus they enhance security. Probably the most common use case for filters is obtaining user consent before executing specific actions. This can be achieved using the function invocation filter, which provides a way to intercept function calls and validate whether they should proceed. The function invocation filter is based on the `IFunctionInvocationFilter` interface, allowing developers to implement custom logic before, during, or after a function is executed. This ensures that AI applications adhere to responsible AI principles, such as requiring explicit user approval for sensitive operations. Other types of filters also exist, including the prompt render filter and the auto-function invocation filter.

Demo: Adding Filters

Instead of discussing these filters here further, let's head back to Visual Studio and see the filter in action. So what I'll show you here is how you can bring humans in the loop when working with Semantic Kernel and plugin function calling by intercepting function calls and adding an approval step before the function will actually get executed. As I said before, this is especially useful when you're dealing with sensitive actions, like, for example, changing the price in a system where you actually want someone to explicitly approve the operation before it goes through. This time I'm using the pie management plugin, which for demo purposes I haven't implemented, but I think you can imagine that this



will be able to update the price of a pie based on the ID. In the main code, here's the important part. I'm registering a custom implementation of the `IFunctionInvocationFilter`, and that is the approval filter. This filter acts like a middleware that wraps around the function's invocation logic. Every time that the model chooses to call a function, this filter will be invoked, and it gets a chance to inspect or even block the call before it reaches the actual plugin method. Here's the code for it. The filter is a class that, of course, implements the `IFunctionInvocationFilter` interface, which, as we see here, defines one method, the `OnFunctionInvocationAsync`. In the filter we create an implementation for this method, which will get called when a function is invoked now. The first parameter, the `FunctionInvocationContext`, contains information about the function being called, like its plugin name, the function name, input arguments, and more. Inside the filter, I'm checking the function being invoked. I checked the name of the plugin and the actual function name here. If that is the one I'm looking for, so then we pause execution and will in this case prompt the user for confirmation. Only if the user agrees, will we allow the function to proceed. Otherwise, we block it by setting a custom result. Here, this gives us complete control over whether or not a function is allowed to run, and we didn't even have to modify the plugin. The AI still gets to reason about when to use a function, but the human now remains in control of the execution. All right, let's narrow this and let's say change the price of Pie with ID 1 to 15. The pie management plugin is found to be needed, but now the filter kicks in. Since there is a match, and we indeed get that prompt. If you say yes, it will just go ahead, but if we say no, we'll also get back a custom error message blocking the further execution of the function.

Using OpenAPI Plugins

The last topic I want to touch on here in our discussion around plugins is OpenAPI plugins. I'm sure most of you are familiar with the OpenAPI specification. It's a way to describe an API so that other tools can connect with it. So it's really useful when building integrations. In many cases, an OpenAPI specification already exists for a given application, offering a nicely structured description of an API's functionality, including the available endpoints, the required parameters, and the responses that we can expect. Now, by leveraging OpenAPI all necessary details about an API's behavior are documented and can be directly consumed by AI models. So indeed, this allows the AI to use the API as a plugin, enabling it to call functions dynamically based on the user's request. For an AI system to interact with an OpenAPI-defined API, a base URL is required to invoke API calls correctly. This ensures that the model can send and receive real-time responses, again, enhancing its capabilities beyond static knowledge.

Demo: Using OpenAPI Plugins



As a final demo in this module, let's look at adding plugins from an OpenAPI specification. What I'm going to do is bring in a real external API into our AI Agent's workflow with just a few lines of code. You can find the code in the `UsingOpenAPI.cs` file. And let's focus here on the `ImportPluginsFromOpenAPI` method. As the name indicates, what I'm doing here is importing an OpenAPI plugin, exposed through a Swagger specification available on this address [here](#). This Swagger file describes the structure, the endpoints, the parameters, and expected responses for our pie-related API. This set of parameters prevents naming conflicts when dealing with complex payloads by prefixing nested property names. Let's take a quick look at the API, which lives in this separate project [here](#). It's an ASP.NET Core minimal API, and it describes a few endpoints to get all pies, get a pie by ID, and also add, update, and delete pie is supported. Once the plugin is imported, it behaves like any other Semantic Kernel plugin. The result is a fully functional agent that can answer questions or take actions based on live API data without us having to manually write wrapper functions or hard-code logic. Let's run this and see if we get the expected results. Let's make sure that we now start the two projects, the API, as well as the application that we've been running so far. If we now simply ask for all pies, we'll get back a list of pies we had in the API. So, there we go. If you have an OpenAPI spec already available, it is very easy to bring that functionality into the functionalities covered by the AI model, as we see [here](#).

Using Semantic Kernel in a Real-world Blazor App

Module Introduction

So far in this course, I have used console applications to show you the different features we have covered. Now I know most people don't build console applications on a daily basis, but it helps us to focus on what we have to learn about. Now in this module, however, I'm going to show you how we can use Semantic Kernel in a real-life scenario inside a real Blazor application. This module is just demos, showing you how I've added the different functionalities we've covered already, but now as said in a real application.

Demo: Exploring the Code

Let's jump straight into our first demo. What we're seeing here is essentially, well, the completed version of the application that we'll be looking at throughout this module. It is the Bethany's Pie Shop Blazor app built specifically for this course, and it primarily contains several administrative interfaces alongside a couple of public-facing pages. And I won't be diving deep into the Blazor aspects



of the app. There are other courses, including Blazor Fundamentals, that already cover that in detail. And that's actually where this application also got its foundation. Here our focus will be just on the parts that will tie into Semantic Kernel. Once I sign into the app, you'll notice that several administrative sections become available in the menu. We'll begin by looking at this Pie Overview page. And shortly I will show you the code-behind so that you can get a feel of how the project is structured. It's not complicated at all, so no worries there. The Pie Overview page displays a list of pies that is coming from the database, and we can see the details for each of these. So the goal of this first demo is showing you the application and its general structure. Looking at the code, the `PieOverview.razor` file is a Blazor component which is only accessible to users signed in with the administrator role, and that is what you see here. The key section of this component is this very simple loop, this `foreach` loop, which iterates through all the pies in this `pies` list, and it renders each one as a new row in a table. There's also a button that links to the pie details. That's what we see here. In the corresponding component class, that is the `PieOverview.razor.cs`, we'll see that list of pies, which is defined here, and a `PieDataService` instance is injected using the `Inject` attribute, which is the typical pattern for injecting dependencies into Blazor components. Then in the uninitialized `async`, that service is used to fetch the list of pies from the back end and then populate the `pies` collection. The `PieDataService` itself is defined through the `IPieDataService` interface, which lists all the typical operations you'd expect, which is retrieving pies, getting pie details, and then adding, editing, and deleting pies. Its implementation, so that is the `PieDataService`, in turn, depends on an `IPieRepository`, which is injected through the constructor and used to perform the actual operations. That repository, so the `PieRepository`, relies on Entity Framework Core to interact with the database via the `Application DbContext`, which is injected here. The method `GetAllPies` uses that context to fetch the list of pies asynchronously, while the `GetPieById` retrieves a pie, a specific pie, using its ID. The methods for adding, updating, and deleting pies are very straightforward and follow usual EF Core patterns. All of this works seamlessly, thanks to dependency injection, and that is actually wired up in the `Program.cs`. There I'm registering the repositories and the services, some of which we'll use later as part of the startup process. So, there we go. That is the general structure of the application. Now it is time to explore how to use Semantic Kernel in this set up.

Demo: Registering Semantic Kernel

We have now seen the application and its general structure. And so now I'm going to add Semantic Kernel into the application, making sure that the necessary packages are included and configuring everything properly so that we're ready to use Semantic Kernel in the upcoming demos. As you've seen before in the cron-job applications, getting started with Semantic Kernel simply



involves installing the right NuGet package. Here's a project file in which I've added the latest available version of the Semantic Kernel package as of the time this course was recorded. Because we're working in a web context specifically inside of a Blazor application, it makes the most sense to centralize our setup in the Program class. I'm using the AddKernel here to register Semantic Kernel with ASP.NET Core's built in dependency injection system, and this allows us to inject it wherever we need it later on in the app. Right after that, I'm chaining to AddOpenAI methods calls to configure both chat completion and image generation features. Now, since this is a web-based setup, the settings are stored in the secrets.json file. And within that file, we've got a ModelSettings section that contains values like TextModelName, ImageModelName, and OPEN_API_KEY, of course. I'm storing those values in my model settings, and by doing that, I'm telling Semantic Kernel to work with the OpenAI models for both types of functionality. In the lines you see here, I'm then retrieving the values for those settings. And that's really it. Our app is now already fully configured and ready to make use of Semantic Kernel throughout the rest of the demos.

Demo: Generating Content

Now that Semantic Kernel is configured and you know how the application is structured, it's time to start bringing in the actual AI features. And we'll begin by exploring what capabilities have been added to the Add Pie page. Adding new pies, or more broadly, new content for our site, can be a challenging task. Not everyone is born a writer, and the folks at Bethany's Pie Shop who manage the website would certainly appreciate a little bit of help. So, let's take a look at the Add Pie page here in action. Let's say that a new blueberry cheesecake has just arrived in stock. That's the input we enter here, so just the name of the pie. From there we can optionally enter a short description ourselves, or we can generate one automatically. Of course, we could open a tool like ChatGPT, but it is far more convenient to stay within the context of the application and have the content generated here right on the page. So I click this Generate button, which triggers OpenAI access through Semantic Kernel to produce a description based on the input and a prompt that I defined behind the scenes. And just like that, a nicely worded description appears. The same goes for this long description. Both fields have a character length expectation set, and that is taken into account when generating the new content. Instead of waiting for a professional photographer to take product images, why not use the image generation to create something, again, automatically, without leaving the application? The image will get shown here right in the UI. How cool is that? Once the price is filled in, we can submit the form, and the new pie will appear in the list of pies. So, how is this all implemented in code? Let's take a look. The PieAdd component is a standard Blazor edit form with a few input text fields for entering the pie details. In addition, the component defines several buttons, which are tied to event handlers, which live in the code-behind class which I'm showing just a



second. The form is bound to a Pie model instance, which captures both the user entered and the AI-generated input. In the Component class, we inject the kernel. We had already registered the kernel using this AddKernel method in the Program.cs, and behind the scenes, that is actually a transient registration. During the component initialization, so in the OnInitialized, we use the kernel to get access to the chat completion and image generation services, which were added as part of the Semantic Kernel setup. Let's look at how the whole description is generated. The code should actually look familiar. We prepare a well-structured prompt that gives the model clear guidance and define the maximum content length right inside of the prompt. Then we stream the response as it is being generated. And this works beautifully in the UI as we've seen because Blazor updates the input field in real time. I'm using the await foreach to collect the generated text as it comes in, and I append it to the existing value of the Pie.Description property. I'm also calling the StateHasChanged to make sure that Blazor refreshes the UI while the content is being populated. The logic for generating the long description is nearly identical, with the main difference being the increased length of the expected output, so set here to around 100 words. Finally, there's the image generation part, and again, this code should look familiar. First, we clear out any previously generated image and reset the URL. Then we prepare a descriptive image prompt to something that tells the model exactly what kind of image I want to generate. We also configure image generation options, which we set up earlier. And then the call the GenerateImageAsync passes in the prompt and the options. And after a short wait, we receive a stream of image bytes. We save those bytes as a file on disk, although you could easily store it somewhere else if needed. Then we create an HTML string called GeneratedImage and display it in the UI using the markup string, which renders raw HTML to show the thumbnail. The Pie.ImageUrl property is updated with the GeneratedImage part so that when we hit the Submit, all of the pie's data, description, image, and price is ready to be passed to the service and repository to save it into the database. And just like that, we've introduced an AI-powered way to help content authors create new entries without ever leaving the actual environment. That could not have been easier. Now let me show you one more thing here in this demo. I say an alternative way to work with the kernel and dependency injection in an ASP.NET Core application. So far I used this kernel that I injected directly into the component, but I have an alternative option. I can create another class that really wraps the kernel and is also responsible for its configuration, which we then do directly in this class too. That's what I have here in this KernelService that we're looking at here. I'm getting an instance of the model settings, that is this class here, which is a representation in code of the settings that we saw earlier in the secrets.json. In the InitializeKernelAsync method, I'm then setting up the kernel using the CreateBuilder. I'm adding again chat completion and image generation, and then I'm using that instance of model settings that was passed in to get access to things like the API key. I then return. The initialized kernel. If I use this approach in my Program.cs, I just register this new service now, as we see



here. And then let me show you an updated version of the PieAdd class. Instead of injecting the kernel directly, I'm injecting the KernelService, or which I now invoke the InitializeKernelAsync here in the OnInitializedAsync, and that gives me access to the kernel. I can then use the kernel, for example, here in the GenerateShortDescription to get in a streaming way that short description. Now while the net result of this approach is the same, I get access to the kernel in the component, it does offer us a clean way to put all the kernel setup code in a separate class which we can then pass around and inject. Now, I'll stick to what we used first for the rest of this course. I said the net result is the same, but know that this, too, is a common and interesting way to work with the kernel.

Demo: Doing More with Prompts

Next to general creation of content, we can use Semantic Kernel for much more than just have a talk with a model to generate content like text or images. Let us see in this demo what other options we have here. Here is the running application again. In the public part of the application, it is possible for a customer to create a support ticket. I can go here and enter a message like a complaint, as you'll see here. Let's enter "I ordered a cheesecake but received a strawberry pie. I want this replaced!", which is a complaint. And let's submit that. Now, behind the scenes, I've asked to summarize the message thread in one sentence for the support agent to more easily see what is going on. And that is what we see here. And if we go into the details, we can also see that the sentiment isn't very happy, which is shown by this smiley. Now both the summarization and the sentiment analysis was done by the AI model. Now there's another feature I want to show you here, which I'll show you in the code. Go back to the Contact Support, and I'm going to enter now a message in Dutch, so in another language. I've entered the message in Dutch, which is basically saying that I received the wrong pie and that as a customer I am not very happy. I submit it. And if we now look at the ticket overview, we see an English summary, and if we go again to the details, we see the Dutch message, as well as a translated message. So what I've done here is now also translating the message through the AI model. Let's see how I've done all that using Semantic Kernel in combination with OpenAI. I'm not going to show you the UI Blazor code. Let's look at this class here, where all this functionality lives, which is the TicketDataService class in the Services folder. In the AddMessageToTicket methods, I'm calling a few other methods, each doing one of the functionalities we just saw. Let's look first at the GetTicketMessageSentiment method, in which I pass the message entered in the ticket. I'm using a prompt here, which will ask to analyze the sentiment of the entered message. The prompt is built up using a prompt template, as we can see here. There are definitely multiple ways to do this, as we saw. Here I need to add the value of the ticket message, and so I'm using a template with an argument. The prompt is very explicit on what to return. So that the model won't give me back a whole sentence, but instead just positive,



negative, or neutral. And based on that value, I'll show a different image, a different smiley in the UI. I could have also just concatenated the prompt, which I'm doing here in the language detection method. In this method, I'm again using the entered message of the user, and first I'm asking the model to detect the language and give me back the two-letter code for the detected language. The second prompt is created to translate to English and just give me back the translated content. Nothing else. In this code, I'm then using Semantic Kernel to first detect the language, and if it is not English, so not EN, then I'm continuing to execute a second prompt on the model, this time to get back the translated content. And I save all that on the message. Finally, I also need that summarization support for my message thread, and that is done in this SummarizeTicket methods, which, as we can see, has quite a long prompt. And for simplicity, I've again used string concatenation, but we could have worked here too with prompt templates. In this case, I have a little bit of functionality that inside of the string. Here I'm checking if the pie and the name is an empty string, and otherwise, I use not specified here. And here I'm invoking a method to concat all messages from the thread of messages of a given ticket, since that is what the AI model will need to summarize. In this part of the prompt, I'm being very explicit about what the AI model should be returning to me, and that is just JSON containing these three summarizations, one with a max of 30 words, another one as a ticket title, and then one final one that captures in 1 word the satisfaction scores out of this list of strings that I have specified. So, based on what the model thinks about the message. That result, so the JSON, is then passed, and the different parts are extracted and stored in the ticket. So, there we go. We can create much more complex prompts, optionally including templating support that we have in Semantic Kernel to have the AI model do much more than just creating some content.

Demo: Using Plugins

Let's see how we can extend the functionality of the site using the combination of an AI model and plugins. And to show you this, let's look at the home page. I am logged in here so the site knows who I am. One of the other properties I keep in the database is the address of the user. Based on this information, the site displays two items, the nearest store location based on the location of the user, and the weather at that location, which is what we see here. This is working thanks to the use of plugins. Let's take a look. As you know by now, the model doesn't know anything about the current weather, and the location of the nearest Bethany's Pie Shop store is also information that the model can't really, well, know. Here is the StoreLocation component, which is what we saw in the demo. We get the kernel injected, and on that kernel instance I'm importing two plugins, the StoreLocation plugin and the weather plugin. Both are classes located in the Plugins folder. The StoreLocation plugin defines a single function marked as KernelFunction, which means that under normal



circumstances, Semantic Kernel will advertise this to the AI model upon making a request, saying that indeed it knows how to find the closest store based on the current location. This is, of course, a simplistic implementation, but you get the idea. The second plugin is the WeatherPlugin, which returns accurate weather information on the location we pass it. This, too, is marked as a `KernelFunction`. And of course, in a real life situation, we could use here an external weather service. Back in the Blazor component, indeed, we see that I have imported these two plugins and I've also defined settings saying that the `ToolCallBehavior` should be `auto` so the model can freely decide to use these plugins when it deems it necessary to do so. This part of the code is Blazor code to access the current user, and we also get access to the city of the user here. Next, I'm using a first prompt, where I ask the model to find the nearest Bethany's Pie Shop store, which I'm pretty sure the model can't know. So executing this prompt, which I trigger here using the `GetChatMessageContentAsync`, will result in the model asking to execute the local function. And it is Semantic Kernel that handles all that for us. So it invokes the method with the correct parameters it gets from the model, returns the result as part of the next prompt it sends, and it will get back to the store location. That is displayed, but it's also used as input for the second prompt, which then asks for the current weather at that store location. The flow is similar. It's all again handled through Semantic Kernel. So thank you, Semantic Kernel. We can easily work with plugins to extend the functionality of what the AI model can do for us.

Demo: Adding RAG with Functions

We have seen function calling as a way to expand what a model can do already in the previous demo. And in this demo, I'm going to show you the sales bot inside Bethany's Pie Shop, which is going to help with getting insight into sales data directly from the Blazor application. We're going to extend, augment the prompt, let's say, with sales data retrieved locally. So we're going to apply the concept of RAG here through the use of functions inside of a plugin. Here's the running page where we can ask a question like, please compare the sales of January 2024 with January 2025. We already know that AI models are powerful and know a lot, but they probably haven't been trained on the specific sales data of Bethany's Pie Shop. That's internal data, definitely not something that a public model should be aware of. But looking at the response, it does seem like the model knows our sales numbers and has generated a meaningful answer based on that data. So, how is that possible? Well, that's again thanks to plugins added through Semantic Kernel. Let's take a look at the code behind all this. So we need to look at the `SalesBot.razor.cs`, which is the code-behind of the component. There I'm using the `chatCompletionService` again, which I get through the injected kernel. Next, I'm importing on that same kernel instance, also a plugin, and that is the `SalesInformationPlugin`. If you look at that here in the plugins directory, we see that it is again a regular class, which in this case gets in



an `IOrderDataService`, which is registered here in the `Program.cs` with its implementation. In the plugin, I have a function that gets the sales information again and it's marked as a kernel function. I've added a complete description, which is sent to the AI models so that it knows we can deliver this functionality. The function knows how to fetch the number of pies sold for a given month and year. Remember that in our running application, we ask the model to compare sales between January 2024 and January 2025. So we're using a simple LINQ query to look up the orders, sum up the pie counts, and return the total number of pies sold. If there are no orders found in the specified month and year, it returns -1 as a signal there is no data available. And I've also described that here in the description. Back in the Blazor component, I defined an `OnEnterQuestion` event handler, which is triggered when we entered our sales-related question. It might be that the model is able to answer the question directly, but often it'll need help, and that's where our plugin comes in. I'm creating a `ChatHistory`, adding `assistantMessage` explaining which persona the model should adopt. Then we simply call the well-known `GetChatMessageContentAsync`, passing in the history, the settings, and the kernel. I think I haven't pointed out the settings, but indeed, as we hear, that is set to automatically invoke any function the model deems necessary. And so that will happen when the model creates the response. It sees it doesn't know about our sales data, but we have a function that exposes this functionality, so we invoke that. We are retrieving data, and that data is sent as part of the new prompt that is going to the AI model. The prompt is augmented with extra data so that the model can create an enhanced response. So we're definitely applying right here, and this happens thanks to the automatic invocation of a function.

Demo: Adding RAG

RAG can be applied in several ways. We just saw the use of a function which pushes extra data onto the prompt, but sometimes we can decide that we'll always want to include data with the prompt. So we'll use a very static approach here to also use RAG. Take a look at this page here, the Pie Creator, which I have applied the principle of RAG. Let's say that we have some ingredients lying around, and we want to look in the list of pie recipes which pie we can still create with that. I've pasted in some ingredients, and we'll ask the model to figure out which pies can still be created, if any. And it found correctly that a pumpkin pie can still be created. If I change this to one egg, it'll correctly say that it cannot create a pie. It can almost create a pumpkin pie, but not in full. Now how did I do that? Well, I've applied another basic form of RAG here. Let me show you. In the database, I have through data seeding added a large list of pie recipes, including their ingredients. What I want is that the model will search in these ingredients for a possible match. If you think about it, this is different from what we've been doing with function calling. Here, the model is not going to call an external function to fetch some data. We are actually going to feed the model all of the



data it might need upfront. So we are going to retrieve the data ourselves from the back end and augment the prompt that we send to the model with all of that data. Hence, definitely also retrieval-augmented generation. Indeed, in the Pie Creator, I first retrieve all the data through the data service. In the prompt, I'm clearly explaining to the model via Semantic Kernel what it should do. It should search through the list of pie recipes and see which pies can still be created using the ingredients that are passed in. But, and this is the augment part again, I'm going to include my entire database of recipes as part of the prompt. That means that the model now has all the context it needs in a single prompt. From then on, it's entirely up to the model to figure out if it can find a matching recipe based on the ingredients that we provided. This is a simple, but effective way to create a rank-based solution using Semantic journal.

Diving into Advanced Topics with Semantic Kernel

Understanding Embeddings

We've now seen practical ways to use Semantic Kernel in a real-life application, but there's more to discover in the framework, which I think are definitely already more advanced topics. In this module, I have grouped several more advanced concepts together, including embeddings, memory, agents and logging. And let's take a look at this in detail. And we'll start with the exploration of embeddings, a concept used commonly by LLMs, but not one that we have interacted with directly so far. So what are embeddings, really? Well, they are a way to represent text as numbers so that machines can process and understand a language in a more meaningful way. Instead of treating words just as sequences of letters which computers don't understand, embeddings capture the relationship between words, their meanings, and how they are used in different contexts. So, okay, how does that work, then? Well, imagine a vast space where each word is placed based on its meaning. Words with similar meanings, like dog and puppy, are positioned closer together, while words with different meanings, like dog and airplane, are much further apart. This high-dimensional space helps AI models to recognize patterns, understand context, and make better predictions. Using embeddings, AI systems can improve search accuracy and provide better recommendations and thus enhance natural language understanding. They form the foundation of many AI applications, including chatbots and translations and semantic search. So yes, without knowing, embeddings have powered a lot of what we have used already so far under the covers. So let me show you this in a bit of a graphical representation where we use the word like dog and the AI model doesn't process it as plain text. Instead, it converts it into a numerical representation known as a vector



using an embedding model. And this vector is a set of numbers that places the word in a high-dimensional space. The key idea behind embeddings is that words with similar meanings are placed close together in this space. So here's an example with dogs and puppies. Dogs will typically be near puppy because they share semantic similarities. Cats will also be relatively close, since it is another type of animal, but slightly further away. However, if we look at the word airplane, it will be positioned much further because it has little in common with animals. This spatial relationship between words is how AI models understand meaning. Instead of interpreting words based purely on dictionary definitions, they analyze their relative distances in this embedding space. This allows AIs to grasp context and relationships, which is why it can suggest relevant words or understand queries even when exact words are not used. And before we continue, I want to avoid confusion between embeddings and tokens, which we've already talked about before. When working with AI models, it's important to understand the difference between tokens and embeddings, as they play different, but related roles in language processing. So tokens are the building blocks of text that the model processes. When you input a sentence, it gets split into tokens, which can be individual words, subwords, or sometimes even characters that depends on the tokenizer. For example, the words "Bethany's Pie Shop makes the best pies in the world" might be tokenized into separate words or broken down further into smaller subword units. Tokens are what the model reads and generates during text processing. Embeddings, on the other hand, are the numerical representations of these tokens. So once a word or token is processed, an embedding model converts it into a vector, which is that series of numbers that represents the meaning of the word in that high-dimensional space. So this is how AI models understand the relation between words. As we just saw, words with similar meaning like dog and puppy will have embeddings that place them close to each other in this space, while unrelated words like the dog and the airplane will be further apart. So tokens are how language is structured for processing, while embeddings are how meaning is captured numerically. And together they allow AI models to analyze, understand, and generate text in a way that feels natural to us. I have already mentioned that embeddings are used all the time when interacting with AI models. So we have used them without really knowing that we did. In fact, embeddings play a crucial role in how AI models understand and process the questions we've asked it. Thanks to embeddings, models are able to go further than just word matching, since we can capture deeper relations between words or even entire concepts. So, embeddings enable semantic similarity. This means that words with similar meanings, like we see with dog and puppy, are placed closer together in the embedding space. This allows AI models to recognize connections between words, even if they're not exactly the same. And that is super important. In fact, this way, embeddings provide contextual understanding. Typically you have traditional-based keyword search methods, but with embeddings we can capture the context in which words appear. For example, the word bank could refer to a financial institution or a bank by the side



of a river. By using embeddings, AI models can determine the meaning based on the surrounding words. Also, embeddings contribute to efficiency. Instead of comparing words one by one, AI models can perform operations on the numerical representations, making tasks like search and recommendations and text generation faster and more accurate. So in short, embeddings can help with finding related content. That's what it is all about. But we have been searching for years. Think of a SQL query. Well, it turns out that embeddings-based search is helpful in some cases. It's designed to capture the semantic meaning of text rather than relying on exact word matches. It understands relationships and context, meaning that it can recognize synonyms and interpret user intent more accurately. So as another example, if someone searches for AI, an embeddings-based search would recognize that artificial intelligence is closely related, even if the exact phrase isn't present in the data. Additionally, embeddings allow for similarity calculations, enabling more relevant search results based on conceptual closeness rather than just matching words. On the other hand, plain text operates, well, in a more traditional way. It relies on exact word matches, making it less effective when synonyms or variations in phrasings are used. So with an example, if a user searches for AI or a document contains artificial intelligence, a plain text search may not return the relevant result. This limitation makes it harder for traditional search methods to grasp synonyms and contextual meaning. Of course, as we have already said, this type of search definitely also has its merit. AI models use embeddings, but we can also use embeddings for searching with contextual meaning, and I'll show this in the demo very soon. But then comes the question, how do we generate this vector, this embedding? Well, AI models use it internally, but they also expose this functionality for use, using an embedding generation service of some kind. That is what we see being used here from OpenAI, again, through Semantic Kernel, which has this functionality baked in in its abstractions. We can just use the `OpenAITextEmbeddingsGenerationService`, and then on that use the `GenerateEmbeddingsAsync`, passing in the text that we want to generate the embedding for. And this embedding we can then use for semantic search, and that I'll show you in the demo. One more thing before we head to that demo. You may be wondering about the use cases for all this, as it looks a bit further out of the context what we have been looking at so far. The knowledge management is a first one, and it benefits from embeddings by allowing us to index and retrieve documents based on their meaning rather than just keywords. This makes it easier for employees or users to find relevant information, even if they phrase their queries differently from the document's exact wording. Think of answering questions on a large set of knowledge-based documents, for example. Next, recommendation systems. They leverage embeddings to suggest relevant content or products. For example, in an e-commerce site, embeddings help match users with items similar to the one they've shown interest in and that improves the personalization. Another important use case is providing relevant answers in chatbots. Instead of responding with predefined answers, chatbots can use embeddings to understand the user's intent and retrieve the most



meaningful response, making interactions more dynamic and more helpful, and definitely more human. Finally, embeddings enhance memory in AI models. This allows AI agents to recall context from past interactions, maintaining conversation history, and provide continuity in responses.

Demo: Creating Embeddings Using Semantic Kernel

All right, although I have done my very best to explain embeddings in a simple way, it is definitely one of the harder ones to explain and understand. And let's see them in action in the next demo. I'm first going to take us through a code example where we convert product descriptions into embeddings and store them in an in-memory vector store for later use. This is, like I mentioned already in the slides, a common pattern that you'll find in recommendation systems and semantic search and AI-powered product matching. Let's start by looking at the code. The code for this demo is in the `ProductEmbeddingsGeneration.cs`. And in this method, we start by creating an instance of the `OpenAITextEmbeddingGenerationService`, which is a class that lives in the Microsoft Semantic Kernel's connected OpenAI namespace, hence also the using statement here at the top of the file. We pass in the model name and the API key, which is pulled again from an environment variable, just like we've done before. And this service is responsible for generating the actual embeddings for product descriptions. Next, we set up an in-memory vector store using the `InMemoryVectorStore` class. This gives us a place to store the product embeddings we'll be generating. We call `GetCollection` to work with a collection named `productCatalog`. That's what I'm doing here. And then ensure that the collection exists using the `CreateCollectionIfNotExistsAs sync`. Now for this to work correctly with the `Product` class, a custom class I'll show you in a minute, we need to add the correct attributes on its properties. We then simulate a `productCatalog` using a helper method called `GetProductCatalog`. And that returns a list of three sample products. Each one includes a name, a category, and a description, and we'll use that to generate our embeddings. For each product in the list, we generate an embedding based on the product description, assuming that's what we'll want to search on later on as well. I then call the `GenerateEmbeddingAsync`. That's the important thing here. And I assign the resulting vector to the `DescriptionEmbedding` property, a property on the `Product` class. After that, I store each product in the vector collection by calling the `UpsertAsync`. This ensures that the product data, along with its embedding, is added or updated in the store. And once all the products are stored, we put a list of product IDs that are upserted just to verify that everything went as expected. Finally, this code is going to retrieve a product again from the vector store, including its vector. And we do this by fetching the `firstProductId` and call `GetAsync`, requesting that the vector is included in the result. And then serialize and display the product so we can inspect the full record, including the embedding. Let's take a look at the `Product` class quickly. It is a simple data



model class, decorated, however, with the right attributes so that Semantic Kernel's a vector store system knows how to interpret it. The product ID is marked as the key for each record. The name, the description, the category are stored as data fields, and then the DescriptionEmbedding, that's the vector, with 1,536 dimensions, which is the typical size returned by OpenAI's embedding models. Finally, we've marked the category field `IsFilterable`, and this allows us to filter or group products later if needed. So let's run this. After a few seconds, we'll get the output. First we see the Upserted Product IDs, and next I'm retrieving the first product. And as part of the product, we see the embedding, so that is the vector representation of the product with its description. It is, of course, not something that we understand, but it is the item, its location, let's say, in that high-dimensional space. So, using just a few lines of code, I've built a setup that generates semantic embeddings for product descriptions, stores them, and makes them available for later retrieval. So this can be used for use cases like similarity search, recommendations based on meaning, and not just on keywords. Now let's look at the next one, and that is semantic search. And let's look at that using embeddings for a semantic search. Here's a second method. So let's focus on what's different here. We start again by initializing the same `OpenAITextEmbeddingGenerationService` and setting up an `InMemoryVectorStore`. And just like before, we are working with a collection called a product catalog and we ensure that it exists. So all this code is the same as we saw before. Then I'm again using the same `GetProductCatalog` method to generate a list of products. But here's the first small change. Instead of looping sequentially through each product to generate the embeddings, I'm using a `Task.WhenAll` and `Task.Run` to do this in parallel. This speeds up the processing, definitely when we're working with a larger product list. Now once the embeddings are generated, we observe all the products in the vector store, also in parallel, again for better performance. Now technically, it's a bit different, but the result is the same. We could have done this parallel thing also in the first sample. Now comes the new part, searching using embeddings. Let's say we want to find a product that best matches the query "Foldable electric scooter." And we don't just compare strings. No, we generate an embedding vector for the query using the same embedding service. This converts the query in the same high-dimensional space as our product descriptions. If we take a look at the product catalog, we don't see a foldable electric scooter, but look at this one. It is definitely close to what we need. So over here then we perform a vectorized search by calling `VectorizedSearchAsync`, and I pass in the search vector. This will return the product that is semantically closest. This means the one whose description vector is the nearest to the query vector in that high-dimensional space. If that's a match, we print out a product name and description. If not, well then we display that nothing was found. Here's another example where I am searching for the best noise-canceling headphone. It was the same flow. We generate the embedding, search using a vector, and output the result. Let's run this sample now. And yes, the output is as we had hoped. We find products based on the semantic search. So what did we see here? What is this second demo



doing? Just make sure that you understand. We introduced semantic search. So instead of retrieving by ID, we're now retrieving by meaning. We're using vector similarity to compare a natural language query against the content of a product catalog. And that's actually how we found the result. Otherwise, we wouldn't have found that scooter. So what we saw here is how you can create a basic AI enhanced product search that goes beyond keywords. It's able to understand intent and return relevant matches even without exact matches.

Working with Semantic Kernel Memory

In the next part, I want to introduce you to using Semantic Kernel memory. And before I explain Semantic Kernel memory, let's talk about the concept of memory in AI systems. In general, memory in an AI system refers to the capability to store and retrieve information from past interactions with the AI or data. This allows the AI to maintain context. It can remember user preferences and it can thus provide more relevant and personalized and consistent responses. Typically, memory can be implemented through techniques like embeddings, which we just saw, and vector databases, enabling the system to recall information based on semantic similarity rather than just keywords. In Semantic Kernel, memory is a powerful feature that enables AI systems to store and retrieve contextually relevant information. LMs are trained on vast public data sets. We've already discussed that earlier in this course. And because of that, they naturally don't have access to private or application-specific data, like company policies or personal schedules. This is where Semantic Kernel memory can be used. Remember that we already saw that we could use plugins here to access in real-time information and that stays relevant. Anyway, memory in Semantic Kernel works by using embeddings, which we just talked about. When you store information like product details, internal guidelines, that information is converted into an embedding and saved in a vector database or an in-memory store. When the AI system that needs to retrieve that information, the user's query is also transformed into an embedding, again like we just saw when doing manual embeddings. The system then performs a semantic search, comparing the query with the stored embeddings to find the most relevant matches. For example, if a user asks "How do I submit an expense report?", the system can retrieve documents and also mention terms like reimbursement or travel expenses. Again, like we already saw, even if the wording doesn't match exactly. To optimize performance, especially with large documents, Semantic Kernel supports chunking, which means breaking down a long text into smaller, more manageable pieces. Each chunk is installed separately, ensuring that only the most relevant sections are retrieved when needed, while also respecting the token limit of LLMs. So in Semantic Kernel, memory operates by storing, retrieving, and integrating data using embeddings. And data is stored, it is transformed as embeddings, so the vector representation that captures the meaning of the text. When you need to retrieve the data, Semantic Kernel converts the query



into an embedding and then compares that with the stored embeddings. This allows us to find the most semantically similar matches. This integration with embeddings makes memory retrieval much more powerful than simple keyword searches, as it is based on understanding the context and meaning of the information. And to work with the memory in practice, we are going to use kernel memory, which is an external open source library from Microsoft, which is built in a very similar way to Semantic Kernel, in that it abstracts away much of the concepts. Here in this snippet, I'm initializing kernel memory using a `KernelMemoryBuilder`. I'm using two models to initialize it, using two of the extension methods, one for the text generation and one for the embedding generation, which would indeed be used to convert content into embeddings. I'm going to use here the simple vector DB as our vector database, which will run entirely in-memory so that we can focus on the memory part here. And then to the build method I'm passing the `MemoryServeless` type here, indicating that since we had a demo going to host the service in the application itself. Next, our application also needs to ingest data. So we're going to convert texts or documents into embeddings for storage in a vector database. This is actually very simple to do, since kernel memory has several methods we can use based on the content we want to feed it. In the snippet we're calling the `ImportTextAsync` method, passing in the text we'll want to store. This is an abstraction on all the underlying things that are happening. With just this one line, restoring that content. Other methods to store, for example, a document or a chunk thereof are also available. In any case, the result is that this data is now stored in our vector database, which is here in-memory. Next, we'll want to get information. Again, kernelMemory will abstract away unneeded details and make this process also very simple. Instead of performing a manual search and then including the content with the prompt to the LLM, we can just use the `AskAsync` method, passing in our question. So that question to the input is converted and used as input for a semantic search in the vector database, and the results will be combined and sent as part of the actual prompt to the LLM, which will now produce a response based on that. So as you can see, a lot of stuff is happening because of this `AsAsync`.

Demo: Setting up and Using Kernel Memory

Let's take a look at all of this in a full example, where I will also then integrate this in full with Semantic Kernel. Let's kick off the first demo using Kernel Memory. A Kernel Memory is a dedicated library built by Microsoft to make it really easy to implement retrieval-augmented generation in our applications. It's a library, so the first thing I've done is adding the `Microsoft.KernelMemory` packages, as we see here. Now, let's use that. I created this `MemoryService` class, which wraps all Kernel Memory functionality in one place. This class takes in again two parameters, the name of the model we'll use for text generation and the name of the embedding model. Inside the constructor, we begin by retrieving our OpenAI



API key from the environment, and then we configure Kernel Memory using the `KernelMemoryBuilder`. We set up two things here. First, we register the text generation model, which will be responsible for answering the questions, and secondly, I add the embedding generation model, which is used to convert our input text into vector format. Finally, we plug in a simple vector DB. This is an in-memory vector store which is perfect for demos and development purposes. It requires no setup, works right out of the box, which is more than enough for what I'm doing here. Once the builder is fully configured, we call the `build` method, passing in as type `MemoryServerless`, and this basically means that the Kernel Memory will run entirely inside of our application, so no additional services required. Next on this `MemoryService` class, I'm exposing two methods. The first one, the `SaveTextAsync`, takes a string of input text and stores it in-memory. Under the hood, Kernel Memory will automatically chunk that text, generate embeddings for each chunk, and store them in the vector database. Now that is a lot of work that is now done for us instead of doing that manually, which is what we saw in the previous demo. Next is the `QueryMemoryAsync`, and that allows us to ask a question. This is where the magic of RAG now happens. Kernel Memory will semantically search the stored data, retrieve the most relevant chunks, and combine them with the user's question before passing it to the LLM to generate a natural language answer. It can even give us back citations used to generate that response. Now, let's see how we use that service. In the `UsingMemory.cs`, I have the `SaveTextAndAskQuestion` method. In there, I'm creating a new instance of `MemoryService`, so that's our new class, passing in the model names, nothing special there. Then I'm storing two pieces of content, some simple work policy statements. Of course, this could be a lot more, and Kernel Memory also supports that we send an entire document, for example, which it will then pass and store for us. What I'm storing here is when employees can work from home and a second policy about required availability during working hours. This is just text, but this can now be used by the LLM in a rank-based approach. Once that is done, we prompt the user to ask a question, something like, "Can I work from home on Tuesday?" So let's try that and see if the LLM actually understood and used our policies. Let's now ask if I can work from home on Tuesday. And the response will be no, you can't, and that is based on what we put into the memory. And before we continue, let's take a moment to appreciate what is happening here. We didn't write any code to chunk the input text. We didn't manually generate embeddings. We didn't build a prompt that combines the user message with static relevant data. All of that, chunking, the storing, the semantic searching, the combining of context, calling the LMM, is handled by Kernel Memory behind the scenes. And this really simplifies building RAG-powered applications, and it makes it much more approachable to use private data with an LLM in a real app. Now we can go further and use Kernel Memory as a plugin in Semantic Kernel so that we get the benefits of Semantic Kernel's function calling capabilities as well. So, basically treating memory access as a callable skill. We'll use the memory plugin for this. I have in



my memory service added this method here, the `UseMemoryAsync` method. I'm doing the normal Semantic Kernel things, setting up the kernel, adding a chat history, and getting a chat completion service. Nothing new here. Next, I create a `MemoryPlugin` instance, passing in the Kernel Memory service. This plugin exposes functions like the `ask` that we can call directly from our prompt, which is what I'm going to do here. And by importing this plugin into our kernel under the name `Memory`, which is what I'm doing here, make this functionality accessible as part of our prompt execution flow. Now, since it is a plugin, I've also set `AutoInvokeKernelFunctions` behavior so that this memory plugin can be used when needed. Look at this prompt. Thanks to Semantic Kernel, we've got a natural language style prompt that calls into the Kernel Memory, performs a semantic search, and then uses the result as part of a larger prompt execution. We then add the user message to the history and simply call the `GetChatMessageContentAsync` method, as we've done already so many times in this course. That's now use that, and that's what I'm doing here. I'm again storing the same content in memory that hasn't changed. Then we now simply call the `UseMemoryAsync`, passing in our question. Under the hood, this will go through the memory plugin, use the semantic search over our stored knowledge, and return a helpful answer. And the best part, we didn't have to manually perform any embedding, retrieval, or prompt augmentation. Semantic Kernel and the Kernel Memory plugin handled this for us. And Kernel Memory now works in the way we have come to work with plugins already. So it taps into the way this all works with Semantic Kernel. That is pretty cool if you ask me. Let's now run this again to make sure that this approach works as well. I'm answering my same question, and that gives me back a correct response based now on Kernel Memory and its plugin.

Using AI Agents

In the next part, I want to explain to you a bit about what is being touted as the next big thing in AI, agents. At the time of the creation of this course, there's support for agents when using semantic kernel, although the way that it is supported may still change. So far, without really realizing it, we've worked with a single-agent set up. What I mean with this is that we've built an app where one single AI agent handles everything for managing conversations, processing user input, or interacting with plugins, and so on. It's all been handled by this one single central agent. Now, if I say things like this, it means that we can also work with multi-agent systems, and that means we'll need to be in control of creating agents. And that's exactly what I want to explain here next. Let's first try to understand what agents really are. AI agents are really autonomous systems that, again, perceive their environment, let's say, make decisions, and take actions to achieve specific goals. And to do so, agents can leverage AI models to understand language and context. They can use memory to retain and recall information. And they can also use plugins to perform tasks more



efficiently. However, what makes agents particularly powerful is their ability to work collaboratively. In simple scenarios, a single agent might handle everything, which is what we've done so far. But for more complex tasks, multiple agents can work together, each specializing in a specific role. For example, one agent could be responsible for gathering data, another one for analyzing it, and a third one for taking action based on that analysis. This structured collaboration helps divide complex tasks into smaller, more manageable steps. And then increases efficiency and accuracy. So by enabling agents to communicate and share information, AI systems can handle more complex challenges and ultimately deliver more advanced, even humanlike interactions. Here's a way you can think about having different agents working together. Imagine you are tasked with organizing a large corporate event. Now, if you're handling everything yourself, so finding the venue, arranging catering, managing guest lists, organizing transportation, it is a lot of different and separate tasks. You would need to keep track of every detail, from dietary requirements to ensuring that everyone has a way to get to the event. It is stressful, it's complex, and chances are something will be overlooked. You might end up with a venue that's too small, catering that doesn't meet everyone's needs, or transport issues that leave guests stranded. It is just too much for one person to manage effectively. Now think about tackling the same task, but now with a team. You have one person focusing solely on the venue, another one on the catering, someone else is handling the guest list, and yet another is ensuring that transportation runs smoothly. Each person is dedicated to their role, using their expertise to handle that part of the process efficiently. They communicate and they collaborate to make sure that everything is aligned. And the result is a well-organized event where every detail is covered. This is exactly how AI agents work and collaborate. Instead of relying on just one agent to do everything, we can assign specific tasks to specialized agents. Each one focuses on its own role, working together to achieve a larger, more complex goal. It is about breaking down complexity into manageable parts, and that leads to better outcomes. So talking about advantages, collaboration is certainly up there. By assigning specific roles to different agents, they can work together, each contributing their expertise, and solving parts of the problem. Next, modular components. Each agent is designed to handle a particular task, making the system more flexible and easier to maintain. If one part needs to change or needs to improve, you can simply update that specific agent without disrupting the entire process. It is like swapping out a team member with another one with the right expertise, keeping the project moving forward smoothly. And finally, orchestration. When each agent works independently, they also need to communicate and coordinate their efforts. Orchestration ensures that the right agent steps in at the right time. Okay, I hope I convinced you that agents will be important while building AI apps, even though the support for using them in your apps is still, well, let's say limited at the time of the creation of this recording, which is early 2025. To get support in your apps for using multiple agents, Semantic Kernel does have us covered. There are a number of packages which you see here, which are required to be added to our



app. Now, creating an agent is pretty simple. We have a type for this, and that is the `ChatCompletionAgent`. I'm creating one here passing in the name, a set of instructions that explain to the AI model what the agent can and cannot do. This is needed so that it's clear how far, let's say, the reach of this agent will be and where another one needs to take over. I'll show you in the demo how things work when we have multiple of these agents working together. There are 3 other important aspects that we'll need to configure as well. The first one is group chat. This is where agents interact. Think of it like bringing everyone into the same room where they can then communicate. In Semantic Kernel, the agent group chat class is used to represent this conversation space. And it allows agents to pass information, respond to each other's input, and collectively work towards solving the problem. The next one is the selection strategy. This one determines how the system chooses which agent you should speak next. For simple scenarios, you might just let agents take turns one after the other. But for more complex situations, like when agents need to go back and forth to refine their responses, you'll need a smarter approach. This is where the kernel function selection strategy comes in. It uses a prompt to analyze the conversation's history and decide which agent should take the next turn, ensuring that the right agent contributes at the right moment. And finally, we have termination strategy, which defines when the conversation should end. Without this, agents might just keep talking indefinitely. The termination strategy had to set conditions for when the discussion is considered complete.

Demo: Creating a Weather Agent

Let me show you now what that looks like in practice by introducing a simple agent, a weatherbot. While you may not have really noticed, we've been orchestrating everything ourselves. With agents, we take a step back and delegate those responsibilities. Think of agents really as a self-contained AI assistant. You give them instructions, a skill set via plugins, and let them run independently then, handing prompts, invoking functions, and also maintaining context across turns. Before I'm showing you multiple agents working together, I'm going to show you in this demo how we can create a simple agent, and let's have it do things for us. Here is the code I'm using to create and interact with our first agent. We start by initializing a kernel using an `OpenAIChatCompletion` model just like we've done before. Indeed, all of this will look very similar. Next, we create a new `ChatCompletionAgent`. This is the key part. I'm giving the agent a name and instructions. In this case, that is, answer weather-related questions. That's enough context for the LLM to understand the scope of its responsibilities. It is like passing the system message, which we have done already many times. We also pass in a `KernelArguments` instance, where we tell Semantic Kernel to automatically decide which function to call. That's important because agents support function calling behind the scenes, so they need to know how to choose the right one. Here I'm registering



our weather plugin with our agent. This is a simple custom plugin that knows how to look up weather information. As before, this could be based also on a real API. So now the agent has a goal and the tools to achieve it. So it's ready, we can start using it. Now to interact with our weatherbots or our agent, we use a small helper method called `InvokeAgentAsync`. It sends user messages into a shared chat history and then calls the agent's `InvokeAsync` method. This allows the agent to process the history, call any required functions from the plugin, and create a response. The agent takes care of everything. It maintains the conversation context, decides which plugin methods need to be invoked, and replies with helpful answers without having us to wire anything. And then in the console we use this `WriteAgentChatMessage` method to display who said what. Let's run this and take a look at the result. Here's the output, and we can see some interesting things here. The agent works autonomously answering questions we give it properly and politely. We gave it one single instruction, answer weather-related questions, and from then on it handled everything itself for us, understanding the intent, invoking the plugin, and formatting responses in a friendly, helpful tone. And it also keeps track of the conversation history through the chat history. So this is really a single-agent setup, and it kind of looks like we are interacting with some thing, some agent from the outside. Now this lays the foundation for multi-agent systems, where different agents can each handle a specific responsibility and collaborate, and that I'll show you in the next demo.

Demo: Creating Collaborating AI Agents

So yes, we created an agent and with just one agent, you might think, what is the use of this really? We have, in fact, created a single-agent system all the time, right? But now you'll see that it becomes more interesting when we have multiple agents working together in this demo. So I'm going to show you a multi-agent system using Semantic Kernel, where each agent has a distinct role. And the sample we'll use is to have different roles like in a software project team. Just like in a real-world team, it makes sense to divide the responsibilities across multiple agents, each focused on a specific part of the problem. This approach gives us more modular, collaborative, and let's say a realistic way of modeling AI workflows using different agents. Here's the code. And since it's all about Semantic Kernel, I start by initializing a kernel and registering and opening a chat completion model. Then we define three personas, so the different roles for the agents. First, the analyst agent is a business analyst. This agent's role is to interpret the user's request and write clear requirements. The developer agent is a frontend developer who takes those requirements and will build a to-do list app using HTML and JavaScript. And the QA agent is a quality assurance engineer. This agent will check the developer's work against the requirements and will approve the solution if it passes inspection. Here, each agent is instantiated using the `ChatCompletionAgentClass`, and we assign each one their respective



instructions and a unique name, like with the Twitter writer agent. Then I'm bringing them together into an AgentGroupChat, and this is where the real collaboration happens. The group chat allows these agents to exchange messages with each other and with the user in a shared context. So this simulates a conversation among the team members working on a task. And there's also something important that I've added here, a custom TerminationStrategy. And we don't want this loop to go on forever, so we define the ApprovalTerminationStrategy, which stops the conversation once the QA agent says the word approve. And the strategy also limits the maximum number of iterations to 6, just to be safe. Then to kick things off, we feed the group a user message. That is this here, "I need a simple to-do list web app. It should allow adding and removing tasks. Please ensure that it's ready for release." The analyst will take in that message, formulate the requirements, develop and rebuild the app, and the QA agent will run a review. If all goes well, then the QA agent will end the conversation by approving it. As the conversation will unfold, each agent contributes based on their own role and will write the output with clear labels of whom is saying what. Let's run this and we'll see how each agent now plays its role in the conversation and how we'll get to the final result. Here are the results of running this conversation on my machine. So we kick things off with a simple use request for a to-do list web app. From there, our analyst agent outlined clear requirements, everything from UI design and functionality to performance and scalability goals. Next, the developer agent turned that spec into working code, a fully functional HTML and JavaScript app with a local storage, responsive design, and clear instructions. Finally, the QA agent reviewed the result and approved it. This approval triggered our custom termination strategy, bringing the conversation to a close once the QA has given the green light. So what you see here is a coordinated AI workflow where each agent plays a defined role, and together they simulate a real-world product lifecycle, all powered by Semantic kernel.

Adding Logging for Debugging Scenarios

In the final part of this last module, I'm going to show you how we can get more logging information available so that we can make our debugging sessions easier. Before we look at how we can add logging into our Semantic Kernel driven apps, let us first take a minute to understand why it's important. First, logging can help you find issues. If something goes wrong, detailed logs can guide you straight to the root cause. Secondly, observability allows you to monitor different parts of a distributed system. In an AI scenario where we have multiple agents, memory stores, and plugins working together, it is crucial to understand how data flows between the different components. Next, logging is a powerful tool to debug AI interactions. AI behavior can sometimes be unpredictable, definitely when dealing with large language models. By capturing detailed information about prompts, responses, and agent decisions, we can



analyze why an AI choose a particular path and how it processed the information. If you're ever wondering why the AI generated a specific response or why an agent choose one action over another, the logs can definitely provide that context.

Demo: Adding Observability

So, let's head back to the code. In this very final demo, let's look at adding observability and logging to our Semantic Kernel-powered applications. When you start building applications with Semantic Kernel, you'll notice that this becomes really valuable, as AI workflows grow more complex. Now we're going to add OpenTelemetry, which is a powerful standard for collecting diagnostics and metrics across distributed systems, and it is well-supported in .NET applications. And it can also help us with insights when using Semantic Kernel. So first I'll set it up, and to do so, of course I need to bring in the correct package, which I've done here in my project file. Then in the code I'm first defining a resource builder, part of OpenTelemetry, using the default attributes. Using this `AddService` adds a service name attribute, identifying this app as the `SKTelemetryExample` in your telemetry dashboards or exporters. Then this line here enables a special switch to turn on, detailed, so more for both verbose, Semantic Kernel diagnostics. This allows you to get deeper insight into what's happening inside the GenAI stack. This can definitely come in handy when we need to troubleshoot how our AI code is executing and behaving. Next, we create a tracer provider and a meter provider, both wired to the Microsoft Semantic kernel namespace. This ensures that traces and metrics emitted by the Semantic Kernel internals are captured. Both are configured to export directly to the console, which is good for demo or debugging scenarios. In real-life scenarios, you would probably export to a file. Next, I'm configuring a `loggerFactory` using `AddOpenTelemetry`. This hooks into the same resource and exports structured log output. We include scopes and formatted messages for added clarity, and we set the log level to information so that we don't get overwhelmed by noise. Once the observability pipeline is in place, we inject the `loggerFactory` into the Semantic Kernel builder using dependency injection. That's what I'm doing in this line here. Now when we call the `InvokePromptAsync` with a simple question like explaining why the sky is blue in one sentence, we will not only get the model's response, but we also get the telemetry output in the background, showing the flow of execution, API calls, and performance metrics. So I showed it. Let's run the sample. There we go. Lots of output. Let's take a look. OpenTelemetry is giving us deep insight into how our AI call was executed. We can track when our `OpenInvokePromptAsync` functions start and then completed and how long it took, but also we see how many tokens were used. Seventeen for the prompt and 38 for the completion, so 55 in total. That is useful for monitoring performance and estimating cost. We also get the full trace context, including activity IDs and the OpenAI model that was used.



In this case, gpt-4o.

Course Summary

With that, we have reached the end of this course on Semantic Kernel in C# and .NET applications. Let us look at the key takeaways from the course to wrap things up. First, Semantic Kernel abstracts away the complexity of working with AI models, making it much easier to integrate intelligent features into your .NET apps while staying flexible for future advancements. As we have seen, the quality of your results depends on your prompts. Crafting clear, detailed, and dynamic prompts is essential to guide the AI to which generating accurate, meaningful, and useful responses. We've also seen the different options we have here to work with prompts, including the HandleBars syntax. We also explored how plugins unlock the full power of AI, connecting your applications to real-world data and enabling scenarios like retrieval-augmented generation. Through plugins, you can give AI access to fresh, private, or internal data sources, which dramatically improves the relevance of its output. Memory and embeddings add context to AI interactions, allowing replications to move beyond stateless conversations and to its rich, contextual, and human-like interactions. With semantic search and memory storage, your AI can truly remember and work smarter. Looking forward, multi-agent systems open the door to solving more complex problems by letting different AI agents collaborate, each with their own focus and expertise. Your applications can become even more powerful, efficient, and also more modular. And finally, observability is critical. Good-looking and monitoring practices are not just helpful, they are essential to debug, optimize, and trust your AI systems in production environments. By mastering these concepts, you're now equipped to build flexible, extensible, and production-ready AI applications using Semantic Kernel and C#. The future of AI in .NET is wide open, and it is yours to shape it. Thanks for watching this course and bye for now